

NIPT 的时点选择与胎儿的异常判定

摘要

无创产前检测 (NIPT) 通过分析母体血液中胎儿游离 DNA (cffDNA), 可早期筛查染色体异常, 其准确性依赖于胎儿 DNA 浓度水平。本文基于题目所给数据构建并分析数学模型, 从而为个性化检测时机推荐和女胎异常精准识别提供科学依据, 具有重要的临床应用价值。

针对问题一, 首先进行数据预处理: 剔除 BMI 异常值、孕周小数化、合并同孕周重复检测, 并按 BMI 分组; 通过 Pearson 相关性分析发现 Y 染色体浓度与孕周弱正相关, 与 BMI 弱负相关, 但相关系数绝对值较小且散点图呈现非线性趋势, 线性模型难以准确刻画其动态变化; 进而构建 GAMM 模型, 引入非线性光滑函数拟合趋势。接着采用 F 检验进行显著性检验, 结果显示, 各组 R^2 均高于 0.75, 拟合效果良好; F 检验所有 p 均小于 0.001, 表明模型整体高度显著, 验证了模型的有效性与可靠性。

针对问题二, 首先基于问题一 GAMM 模型预测 Y 染色体浓度动态达标孕周, 构建达标概率函数 $P(t)$ 与分段风险函数 $R(t)$, 融合形成期望风险函数以综合评估检测成功与失败情形下的加权风险; 在七个临床约束条件下, 遍历所有可行分组方案, 以总期望风险最小为目标进行全局优化, 最终确定 20.7-29.9 为 14.560 周, 29.9-31.2 为 14.142 周, 31.2-32.6 为 15.282 周, 32.6-34.3 为 17.583 周, 34.3-45.7 为 18.355 周, 实现最小总风险值 2517.35; 进一步通过 $\pm 5\%$ 浓度误差模拟实验, 发现在 $\pm 5\%$ 误差下, 最优方案的总风险波动率低于 3.2%, 而推荐时点的偏移均值 ΔT 不超过 0.8 周, 表明模型具备良好鲁棒性与临床可操作性,

针对问题三, 在问题二实现 BMI 分组最优时点推荐的基础上, 通过 Spearman 秩相关分析筛选出影响达标孕周最显著的前三核心变量: X 染色体浓度、体重、18 号染色体 Z 值, 并引入构建 GAMM 模型以实现更高精度的预测。求解得到的最佳 NIPT 时点: 20.7-29.9 组为 17.861 周, 29.9-31.2 为 16.342 周, 31.2-32.6 为 16.831 周, 32.6-34.3 为 18.053 周, 34.3-45.7 为 18.723 周。同样在 $\pm 5\%$ 误差下, 虽然模型输出稳定性出现小幅度下降: 平均偏移均值 ΔT 扩大至 0.92 周, 总风险波动率 δ 上升至 4.1%, 但其鲁棒性损失处于可控范围, 更贴近临床实际。

针对问题四, 首先对女胎样本进行数据预处理, 构建二分类标签, 剔除末次月经等无关字段并保留 BMI、孕周等 18 项核心特征。系统对比五种学习模型, 其中随机森林表现最为优异, AUC 为 0.8909 且平均准确率达到 0.9445, 显著优于其他模型, 表明其泛化能力更强, 抗样本扰动能力更优, 更适用于女胎孕妇的 21 号、18 号和 13 号染色体非整倍体的精准筛查。

本文构建四阶段 NIPT 优化模型: 问题一用 GAMM 刻画 Y 浓度非线性变化; 问题二在临床约束下优化 BMI 分组与最佳检测时点, 方案稳定 (偏移 < 0.8 周); 问题三引入关键变量提升个体化预测精度; 问题四构建随机森林失败预警模型 ($AUC=0.8909$), 阈值优化后召回率达 84%。模型体系覆盖“规律发现—时机推荐—精准预测—风险预警”, 为 NIPT 智能化提供有力支撑。

关键字: GAMM 模型 F 检验 Spearman 秩相关分析 随机森林

一、问题重述

1.1 问题背景

随着“健康中国 2030”战略的推进与生育政策的调整，我国对出生人口质量和母婴健康管理日益重视。高龄产妇比例上升导致胎儿染色体异常风险增加，使得公众对安全、精准产前筛查技术需求迫切^[1]。

NIPT 作为一种经采集孕妇外周血、分析 *cffDNA* 来筛查染色体非整倍体的技术，因其无创、高灵敏度和高特异性的优势，已成为唐氏综合征等常见染色体病筛查的重要手段。*NIPT* 的检测准确性高度依赖于胎儿 *DNA* 在母血中的浓度，然而研究表明，*Y* 染色体浓度受孕周和孕妇 *BMI* 的显著影响：孕周过早时 *cffDNA* 含量不足，而高 *BMI* 可能稀释胎儿 *DNA*，导致检测失败或假阴性风险增加^[2]。若对所有孕妇采用统一的检测时间，可能影响早期干预窗口。因此，如何依据孕妇个体特征制定个性化的最佳检测时点，并建立适用于女胎的稳健异常判别模型，是提升 *NIPT* 临床效能、实现精准产前筛查的关键科学问题。

1.2 问题要求

针对 *NIPT* 中检测时点选择与胎儿异常判定的关键问题，现结合附件提供的临床数据与实际情况，分析以下问题：

问题一：基于附件数据，分析胎儿 *Y* 染色体浓度与孕妇孕周、*BMI* 等指标之间的相关性及其变化规律。建立 *Y* 染色体浓度与孕周、*BMI* 等变量之间的数学关系模型，并对模型参数的显著性及整体拟合效果进行统计检验。

问题二：根据男胎孕妇的 *BMI* 进行合理分组，结合 *Y* 染色体浓度随孕周的变化趋势，确定每组胎儿 *Y* 染色体浓度首次达到或超过 4% 的最早孕周，将其作为该组的最佳 *NIPT* 检测时点，并分析检测误差对时点确定的影响。

问题三：在 *BMI* 分组基础上，综合考虑身高、体重、年龄等多种因素，对男胎孕妇进行更精细的分类。结合 *Y* 染色体浓度的达标比例、检测误差等因素，建立综合评估模型，为每组确定一个最优检测时点，使得在控制误差的同时，最大程度降低因发现过晚带来的临床风险。

问题四：针对女胎孕妇，以检测报告中的 13、18、21 号染色体非整倍体结果为判定基准，综合 *X* 染色体 *Z* 值、各染色体 *Z* 值、*GC* 含量、读段数、比对率、*BMI* 等多维指标，构建一个科学、稳健的胎儿异常判定方法，提升女胎 *NIPT* 结果判读的准确性与可靠性。

二、问题分析

2.1 问题一的分析

针对问题一，第一小问，首先筛选附件中有效检测数据，剔除 *BMI* 异常值。将检测孕周统一转换为小数形式，对同一孕妇在相同孕周的多次检测取均值合并。依据临床标准按 *BMI* 分组，为分层建模奠定基础。随后，通过 *Pearson* 相关性分析初步探究 *Y* 染色体浓度与检测孕周、孕妇 *BMI* 之间的关联趋势，判断是否具有线性特征。在此基础上，构建 *GAMM* 模型，以 *Y* 染色体浓度为响应变量，检测孕周和孕妇 *BMI* 为固

定效应协变量，并引入非线性光滑函数以灵活捕捉其复杂变化趋势。第二小问，采用 F 检验对所构建的 $GAMM$ 模型进行显著性评估，结合分子自由度和分母自由度判断模型稳健性。

2.2 问题二的分析

针对问题二，第一小问，首先基于问题一 $GAMM$ 模型预测每位孕妇 Y 染色体浓度动态变化，确定其理论达标孕周；进而构建达标概率函数 $P(t)$ 与分段风险函数 $R(t)$ ，融合形成期望风险函数 $E(R, P)$ ，在满足具体临床约束条件下，遍历所有可行 BMI 分组方案，以总期望风险最小为目标，求得最优 5 组划分及对应最佳检测时点。第二小问，通过模拟 $\pm 5\%$ 浓度测量误差，分析其对达标孕周、推荐时点及总风险的影响。

2.3 问题三的分析

针对问题三，第一小问，在问题二 BMI 分组模型基础上，进一步引入原始测序读段数等关键临床协变量构建 $GAMM$ 模型，旨在捕捉影响 Y 染色体浓度动态变化的个体异质性，由于新增变量维度较高，为减少模型复杂度、提升预测效率，需首先对协变量进行科学筛选；本文通过 *Spearman* 秩相关系数法识别核心变量作为后续建模的关键输入。第二小问，为评估模型误差对结果的影响，延续问题二的误差评价框架，模拟多源扰动对推荐时点与风险评估的影响，系统考察模型从“单一扰动”到“复合扰动”下的鲁棒表现。

2.4 问题四的分析

针对问题四，本质为多因素影响下的二分类预测问题，目标是基于孕妇临床特征与测序数据，构建 $NIPT$ 检测失败风险的智能预警模型；考虑到影响因素维度高、类型复杂，本文采用有监督机器学习方法进行建模，选取随机森林、逻辑回归等五种算法进行横向对比筛选综合性能最优的模型；在此基础上，对最优模型进一步实施 ROC 曲线阈值优化，以在敏感性与特异性之间实现最优权衡，为 $NIPT$ 质量控制提供有力支持。

方案整体框架如下：

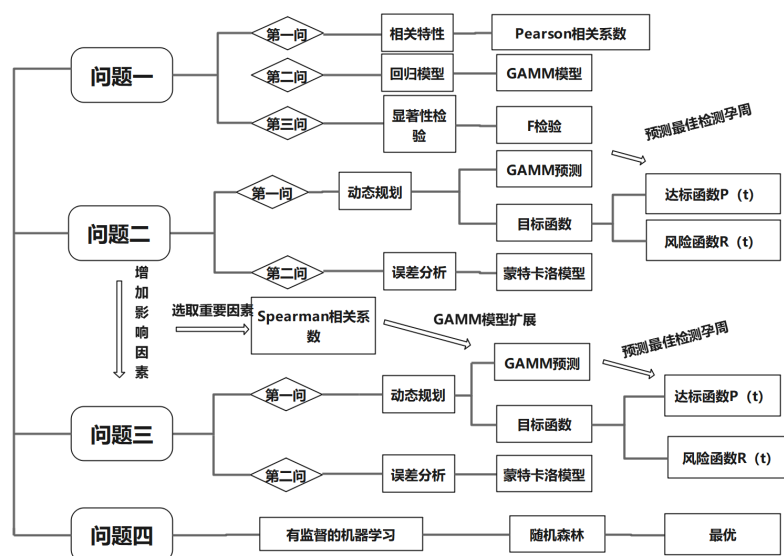


图 1 整体框架

三、模型假设

1. 假设 *NIPT* 检测误差主要来源于技术波动与个体生物学差异，且服从正态分布。
2. *IVF* 妊娠方式（试管婴儿、人工授精与自然受孕）对 *NIPT* 检测结果无显著影响。
3. 检测孕周与实际孕周无显著误差, 即不考虑末期月经。

四、符号说明

符号	说明	符号	说明
r	<i>Pearson</i> 相关系数	β_{1k}	检测孕周平滑函数系数
Y_{ij}	<i>Y</i> 染色体浓度	β_{2k}	<i>BMI</i> 平滑函数系数
$f_1(\cdot)$	检测孕周的平滑函数	ΔT	推荐时点的偏移均值
$f_2(\cdot)$	孕妇 <i>BMI</i> 的平滑函数	p_g	<i>Spearman</i> 相关系数

五、数据侧写

为全面了解数据结构、分布特征及潜在问题，本文首先对原始附件中数据集进行了系统的数据侧写分析，旨在全面描述数据的基本特征、变量分布、异常情况及潜在关联，为后续的数据预处理和模型构建提供科学依据。

5.1 数据集整体结构

原始数据中男胎检测包含 1082 条记录，涉及 267 名孕妇，女胎检测数据包含 605 条记录，涉及 147 名孕妇。男胎数据量显著多于女胎，且每位孕妇平均检测次数较高，表明临床对男胎的监测可能更为密集，检测孕周分布于 $10w+0$ 至 $24w+6$ 之间，集中于 12-24 周，符合 *NIPT* 的常规临床窗口期。数据字段共 23 项，分为孕妇基本信息、测序质量指标、胎儿遗传参数三类。

5.2 孕妇人口学特征

全体孕妇平均年龄为 31.2 岁，年龄范围为 24-40 岁，呈近似正态分布，符合高龄产妇筛查人群特征。平均身高为 161.5cm，体重为 78.6kg，存在右偏趋势。*BMI* 平均为 $32.28kg/m^3$ ，其中 $BMI \geq 28$ 即超重者占比达 62%，表明群体中超重与肥胖孕妇占主导地位。

5.3 测序质量指标分布

为了解核心测序指标的分布特征，本文对其进行了描述性统计，结果如表 1 所示：

由表 1 可以看出，绝大多数样本的测序数据质量良好，满足基本分析要求，然而，部分样本在原始读段数、比对比例及被过滤读段比例等方面存在异常值。

表 1 测序质量指标统计表

指标	平均值	标准差	范围	异常值
原始读段数	4692190.236	948737.953	1342544.000-9895358.000	20(1.8%)
比对比例	0.797	0.015	0.599-0.847	39(3.6%)
重复比例	0.030	0.003	0.021-0.047	23(2.1%)
GC 含量	0.401	0. 003	0.386-0.421	26(2.4%)
被过滤比例	0.023	0.003	0.012-0.038	22(2.0%)

5.4 Y 染色体浓度分布

为初步了解男胎游离 DNA 的释放规律，本文对 Y 染色体浓度进行可视化分析，如图 2 所示：

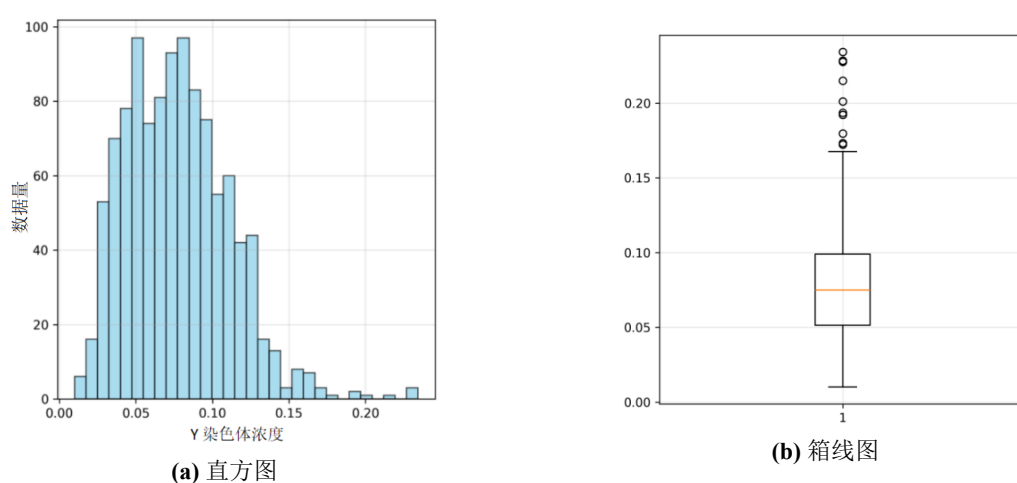


图 2 Y 染色体浓度分布特征

5.5 女胎数据可视化

本文对附件表单中女胎数据进行了可视化，使得数据更加直观，如图 3 所示：

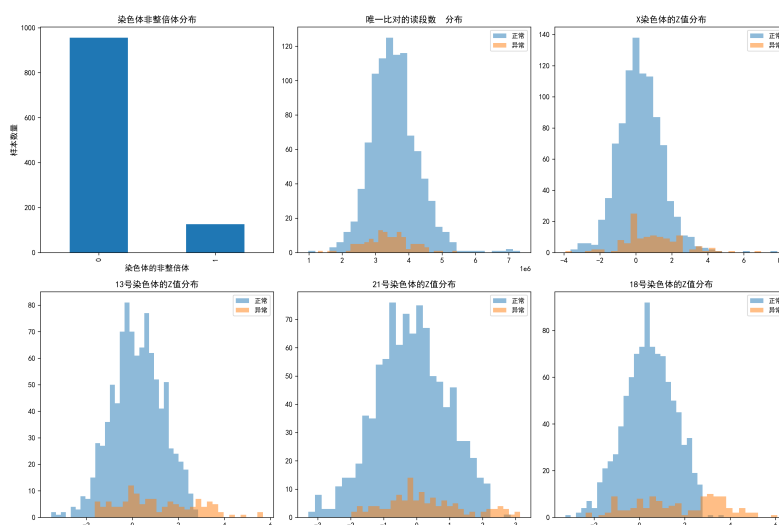


图 3 女胎数据概况

图 2 显示, Y 染色体浓度在 0.05-0.10 区间形成明显峰值, 符合健康妊娠中 *cffDNA* 的典型释放水平; 浓度低于 0.05 或高于 0.15 的样本数量较少, 呈长尾分布特征。进一步通过箱线图观察, Y 染色体浓度的中位数为 0.075, 四分位距为 0.048-0.100, 表明 50% 的样本集中在该区间内。数据整体呈轻微右偏, 上须延伸至 0.18, 其外存在若干高值异常点, 表明存在胎儿染色体异常或检测偏差。从图 3 可以看出, 正常样本的染色体 *Z* 值大多集中在 0 附近, 呈正态分布, 表明其染色体数目在正常范围内。而异常样本的 *Z* 值分布较宽, 部分样本的 *Z* 值偏离 0 较多, 表明这些样本可能存在染色体数目异常。此外, 唯一比对的读段数在正常样本中较为集中, 而在异常样本中较为分散, 可能与测序质量或样本处理有关。

六、问题一模型的建立与求解

6.1 数据预处理

为确保后续建模分析的准确性与可靠性, 本文在数据建模前对原始检测数据进行了系统而严谨的预处理, 具体方法如下:

6.1.1 有效数据筛选

通过查阅参考文献得知 *cffDNA* 检测的准确性高度依赖于测序数据的质量^[3], 为此, 本研究参考该文献中提出的质量控制标准, 设定以下六项指标作为判断检测数据有效性的依据, 如表 2 所示:

表 2 质量控制指标

序号	内容	序号	内容
1	原始读段数 2M	4	被过滤掉的读段比例 5%
2	重复读段的比例 10%	5	GC 含量介于 35% - 40% 之间
3	唯一比对的读段数 1M	6	在参考基因组上比对的比例 70%

仅当某条检测记录同时满足上述所有条件时, 才被视为高质量有效数据, 纳入后续分析。

6.1.2 BMI 异常值删除

编号为 A074 孕妇的 BMI 接近 20, 显著高于群体平均水平且远超孕期合理范围; 而编号为 A160 孕妇的 BMI 接近 20, 显著低于群体平均水平; 二者均属明显异常值, 这里选择剔除。

6.1.3 孕周数的小数化处理

原始数据中孕周以“周 + 天”形式记录。为便于后续建模中的连续变量处理与回归分析, 本文将其统一转换为小数形式:

$$\text{孕周} = \text{周数} + \frac{\text{天数}}{7} \quad (1)$$

6.1.4 同孕周同孕妇检测结果合并

由于部分孕妇在相同孕周进行了多次检测，导致数据中存在重复观测。为避免模型过拟合与样本独立性假设的违背，本文对同一孕妇在同一孕周的多次检测结果进行合并，对所有连续型变量取算术平均值，分类变量以首次检测结果为准。

6.1.5 BMI 分组

为进一步探究 *GAMM* 模型的回归效果，本文将数据根据孕妇 *BMI* 分组进行回归，经误差分析说明回归效果。根据相关文献可知， $BMI \geq 28\text{ kg/m}^2$ 定义为肥胖^[4]，再结合孕妇群体的特殊性，现将孕妇 *BMI* 划为超重组、肥胖 I 组、肥胖 II 组三个组别，如表 3 所示：

表 3 分组情况

<i>BMI</i> 类别	分组名称	临床含义	占比%
[28, 32)	超重组	超重，接近肥胖	51.89
[32, 36)	肥胖 I 组	中度肥胖	39.43
[36, 46)	肥胖 II 组	重度肥胖	8.68

6.2 Pearson 相关性分析

在构建 *Y* 染色体浓度的动态变化模型之前，首先需明确其与潜在影响因素之间的统计关系。本文选取孕周和孕妇 *BMI* 作为核心协变量，采用 *Pearson* 积矩相关系数分析其与 *Y* 染色体浓度的线性关联强度与方向。*Pearson* 相关系数 r 的计算公式为：

$$r = \frac{\sum(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum(x_i - \bar{x})^2} \sqrt{\sum(y_i - \bar{y})^2}} \tag{2}$$

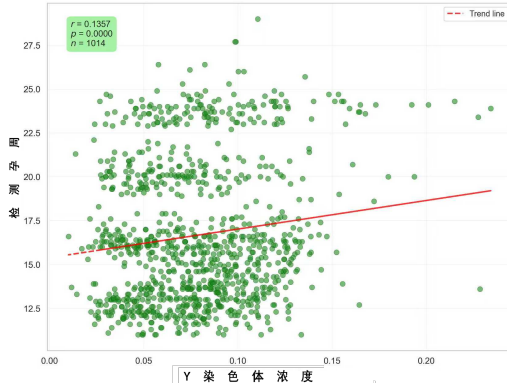
其中 x_i, y_i 为变量观测值, $\bar{x}, \bar{y}$ 为其均值, 分析结果如表 4 所示：

表 4 *Pearson* 相关系数表

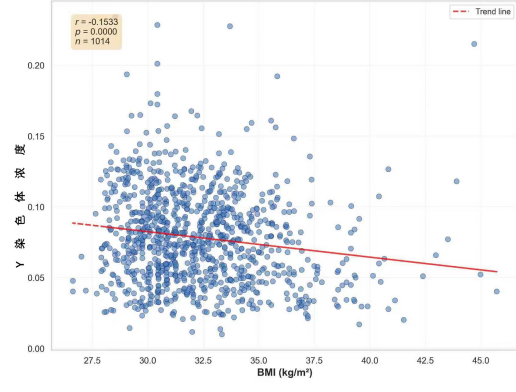
变量 X	变量 Y	皮尔逊相关系数	<i>P</i> 值	相关性方向	相关性强度	显著性
<i>Y</i> 染色体浓度	检测孕周	0.1357	0.001	正相关	弱	显著
<i>Y</i> 染色体浓度	孕妇 <i>BMI</i>	-0.1533	0.001	负相关	弱	显著

由表可知：*Y* 染色体浓度与孕周呈显著弱正相关 ($r=0.1357, p=0.001$)，表明随着孕周增加，浓度总体呈上升趋势；与 *BMI* 呈显著弱负相关 ($r=-0.1533, p=0.001$)，提示高 *BMI* 可能抑制 *cffDNA* 的释放或导致其稀释；孕周与 *BMI* 之间也存在轻微正相关，暗示高 *BMI* 孕妇可能倾向于更晚进行检测，存在潜在混杂效应。

尽管相关性显著，但 $|r| < 0.2$ 且散点图显示关系非线性（前期平缓、中期加速），说明简单线性关系不足以刻画其动态变化。因此，后续建模需引入非线性项以更准确地拟合数据。为进一步探究 *Y* 染色体浓度、*BMI* 与孕周三者间的复杂关系，绘制了二维散点图，如图 4 所示：



(a) 孕周与 Y 染色体浓度的散点图及趋势线



(b) BMI 与 Y 染色体浓度的散点图及趋势线

图 4 Y 染色体浓度散点图及趋势线

图 4 (a) 展示了孕周与 Y 染色体浓度的关系, 二者呈显著弱正相关, 数据点同样表现出显著的正相关趋势, 即随着孕周的增加, Y 染色体浓度呈现上升趋势; 图 4 (b) 展示了 BMI 与 Y 染色体浓度的关系, 结果显示二者呈显著弱负相关, 数据点呈现出明显的负相关趋势, 即随着 BMI 的增加, Y 染色体浓度逐渐降低。二者尽管散点分布较为分散, 表明存在个体差异, 但整体趋势清晰, 验证了 *Pearson* 相关性分析中的结论。

鉴于数据具有非线性趋势、重复测量结构和多因素影响等特点, 本文采用 *GAMM* 模型构建 Y 染色体浓度的动态变化模型。其优势在于使用样条函数拟合连续变量的非线性关系、引入随机效应控制个体间异质性 (如不同孕妇的基础 *cffDNA* 释放差异)、可同时纳入固定效应 (如 BMI、孕周) 与交互项, 模型基本形式如下:

$$Y_{ij} = \alpha + f_1(W_{ij}) + f_2(B_{ij}) + u_i + \varepsilon_{ij} \quad (3)$$

平滑函数展开式:

$$\begin{cases} f_1(W_{ij}) = \sum_{k=1}^{K_1} \beta_{1k} \phi_{1k}(W_{ij}) \\ f_2(B_{ij}) = \sum_{k=1}^{K_2} \beta_{2k} \phi_{2k}(B_{ij}) \end{cases} \quad (4)$$

其中 Y_{ij} 为第 i 位孕妇第 j 次测量的 Y 染色体浓度, α 为截距项, W_{ij} 、 B_{ij} 分别为第 i 位孕妇第 j 次的检测孕周和 BMI 值, $\phi_{1k}(\cdot)$ 、 $\phi_{2k}(\cdot)$ 分别为检测孕周、BMI 的第 k 个 B 样条基函数, β_{1k} 、 β_{2k} 分别为检测孕周和 BMI 平滑函数的第 k 个系数, $u_i \sim N(0, \sigma_u^2)$ 为第 i 位孕妇的随机截距, $\varepsilon_{ij} \sim N(0, \sigma_e^2)$ 为残差项。所到模型有关系数如下表 5 所示:

表 5 *GAMM* 模型参数

组别	α (截距)	β_{1k} ($k=1,2,3,4$)	β_{2k} ($k=1,2,3,4$)
全部人群	0.152	[-0.090, -0.076, -0.053, -0.021]	[-0.427, 0.038, -0.039, -0.080]
超重组	0.147	[-0.146, -0.071, -0.069, -0.059]	[-0.020, 0.001, 0.011, -0.005]
肥胖 I 组	0.185	[-0.176, -0.098, -0.086, -0.089]	[-0.056, -0.027, -0.018, -0.019]
肥胖 II 组	0.153	[-0.098, -0.068, -0.062, -0.057]	[0.121, -0.061, -0.039, -0.059]

为评估 $GAMM$ 模型在不同 BMI 区间对 Y 染色体浓度的拟合效果, 本研究采用决定系数 (R^2)、均方根误差 ($RMSE$) 和平均绝对误差 (MAE) 三个互补性指标进行综合评价, 相关参数如表 6 所示:

表 6 模型性能综合评估表

组别	R^2	$RMSE$	MAE
超重组	0.810843961	0.014741976	0.010592706
肥胖 I 组	0.864018624	0.011514904	0.007345871
肥胖 II 组	0.750419216	0.016200048	0.0123731

各组 R^2 均高于 **0.75**, 表明模型综合解释力较强, 其中肥胖 I 组表现最优, $R^2=0.864019$, 且 $RMSE=0.0115$, $MAE=0.0073$, 预测误差最小, 所以该模型能够有效彰显 Y 染色体浓度随孕周非线性增长及受孕妇 BMI 负向调节的核心规律。为了更直观的查看拟合效果, 本文展示了拟合残差分布图, 如下图所示 (由于篇幅原因只展示全部人群和肥胖 I 组, 其他见附录):

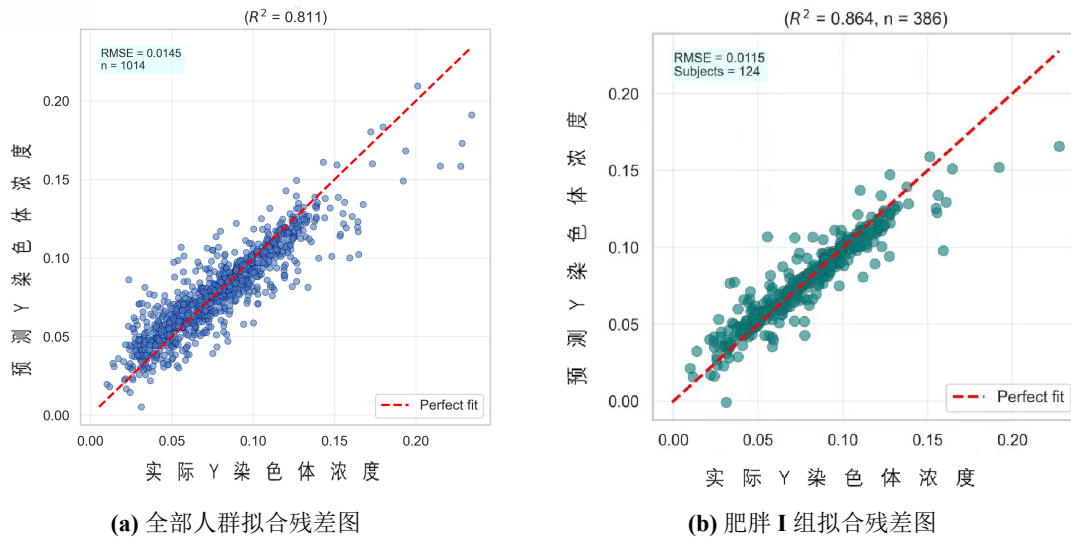


图 5 Y 染色体浓度散点图及趋势线

6.3 模型显著性检验

为评估 $GAMM$ 的整体显著性和关键协变量的非线性效应, 本文采用 F 检验进行统计分析。检验方法如下: 设 M_{full} 为包含所有协变量和随机效应的全模型, $M_{reduced}$ 为不含待检验项的简化模型。提出假设:

$$\begin{cases} H_0: & \text{待检验项对因变量无显著影响} \\ H_1: & \text{待检验项对因变量无显著影响} \end{cases}$$

F 检验统计量 F 计算公式如下:

$$F = \frac{(RSS_{reduced} - RSS_{full}) / (df_{reduced} - df_{full})}{RSS_{full} / df_{full}} \quad (5)$$

其中, RSS_{reduced} 、 RSS_{full} 分别为简化模型和全模型的残差平方和; df_{reduced} 、 df_{full} 分别为简化模型和全模型的残差自由度。分子部分表示由于引入待检验项而额外解释的变异, 分母部分是全模型的均方误差作为误差方差的估计。

在 H_0 成立时, F 统计量渐近服从自由度为 (df_1, df_2) 的 F 分布, 其中:

$$df_1 = df_{\text{reduced}} - df_{\text{full}}, \quad df_2 = df_{\text{full}} \quad (6)$$

给定显著性水平 $\alpha=0.05$, 若计算得 $p<0.05$, 则拒绝 H_0 , 认为待检验项对 Y 染色体浓度具有显著影响, 模型结果如表 7 所示:

表 7 F 检验模型结果

组别	F 统计量	p 值	分子自由度	分母自由度	显著性
全部人群	390.194	0.001	11	1002	极显著
超重组	237.1941138	0.001	9	498	极显著
肥胖 I 组	265.4538373	0.001	9	376	极显著
肥胖 II 组	30.06718722	0.001	9	90	极显著

由上表可知, 在全部人群中 $GAMM$ 表现出极强的统计显著性, F 统计量高达 390.194, p 值 <0.001 , 表明模型解释的变异远大于残差变异, 拟合效果优异。进一步分层来看, 三组的 F 值均远大于 1, 表明该模型解释的变异远大于残差变异; 分子自由度 $df_1=9$, 对应模型中固定效应与光滑项的总有效自由度, 分母自由度 df_2 与样本量一致, 说明误差估计稳定; 所有 p 值均小于 0.001, 极高度显著。综上所述: 在所有 BMI 区间, 所建 $GAMM$ 模型均能显著解释 Y 染色体浓度的变异。

七、问题二模型的建立与求解

本问旨在通过对男胎孕妇按 BMI 进行合理分组, 并确定每组的最佳检测时点, 使得所有孕妇的潜在风险总和最小。该问题本质上是一个带约束的优化决策模型, 需在保障检测准确性的前提下, 权衡“尽早检测”与“确保 $cffDNA$ 浓度达标”之间的矛盾。

7.1 总期望风险最小化

设将孕妇按 BMI 划分为 k 组, 每组对应一个推荐的最佳检测时点 $t_i (i = 1, 2, \dots, k)$, 并假设第 i 组包含 n_i 名孕妇。对于每一名孕妇, 其在特定孕周进行检测所面临的潜在风险可通过期望风险函数 $E(R, P)$ 来量化, 该函数综合了检测成功与失败两种情形下的风险加权平均。因此, 模型的优化目标为:

$$\min_{k, C} \sum_{i=1}^n E(R, P) \quad (7)$$

其中 $C = \{t_1, t_2, \dots, t_k\}$ 表示聚类中心, k 为分组数量, n 为样本总数。

7.2 期望风险函数的构建

为全面评估不同孕周进行 *NIPT* 检测所带来的潜在临床后果，需建立一个能够反映诊断时机与干预窗口之间关系的风险量化体系。

7.2.1 当前检验风险函数 $R(t)$

首先，定义当前时点风险 $R(t)$ 为在孕周 t 进行检测时所面临的风险。根据临床经验和数据统计，我们将风险分为三个阶段：

- (1) 早期检测 ($t \leq 12$): 此时胎儿发育尚处于初期，即使发现异常也有较多时间进行干预和治疗，因此风险相对较低。设定 $R(t)=1$ 。
- (2) 中期检测 ($12 < t \leq 27$): 随着孕周的增加，胎儿发育逐渐成熟，若在此阶段发现异常，虽然仍有干预机会，但时间窗口变窄，风险相应增加。设定 $R(t) = 1 + \frac{1}{5}(t - 12)$ ，表示风险随孕周线性增长。
- (3) 晚期检测 ($t > 27$): 进入晚期妊娠阶段，胎儿已接近足月，此时发现异常往往难以有效干预，风险急剧上升。设定 $R(t) = 4$ ，表示高风险状态。

具体表达式如下：

$$R(t) = \begin{cases} 1, & t \leq 12 \\ 1 + \frac{1}{5}(t - 12), & 12 < t \leq 27 \\ 4, & t > 27 \end{cases} \quad (8)$$

7.2.2 延迟检测风险函数 $R(t+2)$

考虑到实际操作中可能存在初次检测失败的情况，需要在两周后进行复检，这将导致诊断和干预的延迟，从而增加风险。因此，我们引入延迟检测风险 $R(t+2)$ ，即在孕周 $t+2$ 进行检测时的风险。由于延迟两周后，胎儿发育进一步推进，风险也随之增加，因此 $R(t+2)$ 的值会高于 $R(t)$ ，具体计算方法与 $R(t)$ 相同。

7.2.3 达标函数 $P(t)$

为了科学刻画 Y 染色体浓度达标概率随孕周变化的趋势，本文采用 *Sigmoid* 型函数作为达标函数 $P(t)$ 的数学形式：

$$P(t) = \frac{1}{1 + \exp(-\alpha(\Delta t - \beta))} \quad (9)$$

其中， Δt 为当前检测孕周 (t) 与预期最优检测孕周的时间 (t_p) 差， $\alpha=0.5$ ， $\beta=0$ 。当 $\Delta t > 0$ 时，当前检测孕周大于最优检测孕周，此时 Y 染色体浓度已超过预期水平，达标概率较高，可以认为 $P(t) \approx 1$ 。

7.2.4 期望风险函数 $E(R, P)$

结合当前时点风险 $R(t)$ 和延迟检测风险 $R(t+2)$ ，我们构建了期望风险函数 $E(R, P)$ ，用于评估在孕周 t 进行检测的综合风险。该函数考虑了检测成功的概率 $P(t)$ 和检测失

败的概率 $1-P(t)$ ，具体公式如下：

$$E(R, P) = P(t) \cdot R(t) + [1 - P(t)] \cdot R(t + 2) \quad (10)$$

其中 $E(R, P)$ 为期望风险， $P(t)$ 为成功概率函数， $R(t)$ 为当前时点风险， $R(t + 2)$ 为延迟 2 周后的风险。

7.2.5 约束条件

为了构建一个科学合理的 NIPT 检测时机优化模型，本文设定了以下约束条件，以确保分组的合理性、检测的准确性和风险的可控性。

$$\left\{ \begin{array}{ll} b_0 \leq b_1 \leq \dots \leq b_k : & b_0 = \min(\text{BMI}), b_k = \max(\text{BMI}), \text{确保 BMI 区间连续。} \\ 3 \leq k \leq 8 : & \text{分组数不少于 3 组，不多于 8 组。} \\ n_i \geq 50, \forall i \in \{1, 2, \dots, k\} : & \text{每组样本量不低于 50。} \\ P(t_i) \geq 80\%, \forall i : & \text{每组在推荐检测时点预测成功率不低于 80\%。} \\ 10 \leq t_i \leq 25, \forall i : & \text{最佳检测时点限定在孕 10 至 25 周间。} \\ \sigma_{\text{BMI}}^2(G_i) \leq \sigma_{\text{max}}^2, \forall i : & \text{控制每组 BMI 值的离散程度，保证组内特征一致性。} \\ |\mu_{\text{BMI}}(G_i) - \mu_{\text{BMI}}(G_j)| \geq \Delta\mu_{\text{min}}, \forall i \neq j : & \text{要求异组 BMI 均值差异显著，避免混淆。} \end{array} \right.$$

7.3 模型结果分析

通过系统优化，本文将 BMI 划分为 5 个区间，各组的划分依据、推荐检测时点及关键指标如表 8 所示：

表 8 基于 BMI 分组的 NIPT 最佳检测时点与风险评估 1

分组	BMI 区间	最佳 NIPT 时点	平均预期成功率	平均风险值	最小总期望风险
分组 1	[20.7, 29.9)	14.560 周	96.70%	2.17	2517.35
分组 2	[29.9, 31.2)	14.142 周	97.90%	2.116894	
分组 3	[31.2, 32.6)	15.282 周	93.70%	2	
分组 4	[32.6, 34.3)	17.583 周	100.00%	2.173268	
分组 5	[34.3, 45.7)	18.355 周	92.30%	2.392308	

从结果可见：分组 1-2 BMI 较低，Y 染色体浓度上升较快，可在孕 14 周左右完成检测，成功率高、风险低，符合“早检早干预”原则；分组 3 需稍推迟至 15.0 周，以确保浓度达标，维持 93.7% 的成功率；分组 4-5 由于 *cffDNA* 释放延迟，最佳检测时点显著后移，分别至 17.5 周和 18.355 周。尽管此时平均风险值上升，但检测成功率分别达到 100.00% 和 92.30%，有效避免了因过早检测失败而导致的复检延迟与诊断窗口压缩。在该分组方案下，所有孕妇的总期望风险最小值为 2517.35。

本文将风险与 BMI、孕周数关系做了直观可视化，如图 6 所示：

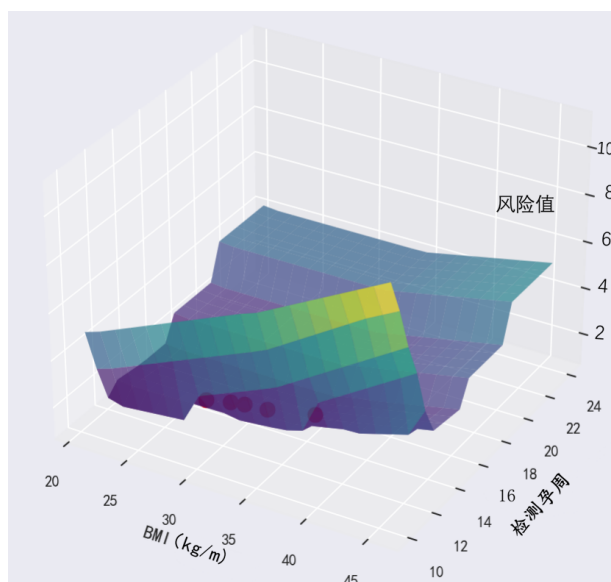


图 6 3D 风险表面

该 3D 风险曲面图直观揭示了 BMI 与检测孕周对 NIPT 失败风险的联合非线性影响：在低 BMI 且早孕周区域，风险值最低（深紫色），表明瘦型孕妇可安全提前检测；随着 BMI 升高或孕周推迟，风险值呈显著上升趋势，尤其在高 BMI 与晚孕周交汇区，风险达到峰值（黄色），反映母体稀释效应与胎源 DNA 动态增长的冲突加剧；中等 BMI 区间风险过渡平缓，适合弹性窗口检测。

为更直观展示 BMI 区间, 本文做了 BMI 分布与最优分组边界图, 如图 7 所示:

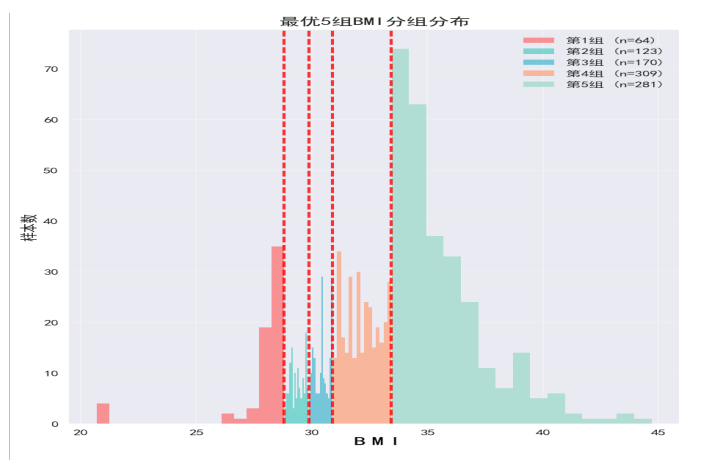


图 7 BMI 分布与最优分组边界图

图中标注了四条的虚线将 BMI 值在 25 至 45 kg/m^2 范围内的样本划分为五个最优分组, 整体分布近似正态但略呈右偏, 表明高 BMI 孕妇比例稍高; 其中, 分组 1 ($n=64$) 样本密度最低, Y 染色体浓度上升最快, 适宜最早检测 (17.861 周); 分组 2 ($n=123$) 密度适中, 浓度增速平稳, 推荐稍晚检测 (16.342 周); 分组 3 ($n=170$) 为样本最密集区, 浓度上升趋势缓, 需进一步推迟 (16.831 周); 分组 4 ($n=309$) 密度高且浓度上升缓慢, 需显著延后检测 (19.053 周) 以规避假阴性; 分组 5 ($n=281$) 虽样本密度回落, 但浓度增速最慢、母体稀释效应最强, 建议最晚检测 (19.723 周) 以确保成功率。

7.4 蒙特卡洛误差分析

为了评估模型的误差及稳定性，这里采用蒙特卡洛误差分析：

7.4.1 浓度扰动

对预测的 Y 染色体浓度添加 $\pm 5\%$ 的高斯噪声 (均值为 0, 标准差为原浓度值的 5%), 模拟实际检测中可能出现的测量误差。基于这些扰动后的浓度值重新计算每位孕妇的个性化达标孕周 $t_p^{(i)}$ 。

7.4.2 时点偏移统计

对每个 BMI 分组内的所有个体，记录其原始推荐的最佳检测时点和扰动后的新推荐时点 t_g^* , 计算每组内新旧推荐时点之间的偏移均值 ΔT 及其标准差 σ_k :

$$\begin{cases} \Delta T = \frac{1}{n} \sum_{i=1}^n (t_{p,\text{new}}^{(i)} - t_{p,\text{orig}}^{(i)}) \\ \sigma_t = \sqrt{\frac{1}{n} \sum_{i=1}^n (t_{p,\text{new}}^{(i)} - t_{p,\text{orig}}^{(i)} - \Delta T)^2} \end{cases} \quad (11)$$

7.4.3 风险波动分析与稳定性评分

计算每个 BMI 分组在误差扰动前后总期望风险的变化率 δ :

$$\delta = \frac{|E_{\text{new}} - E_{\text{orig}}|}{E_{\text{orig}}} \quad (12)$$

其中, E_{orig} 和 E_{new} 分别表示扰动前后的总期望风险值。

定义一个组稳定性指数 S_g 来综合评价各组在不同误差水平下的稳定性:

$$S_g = 1 - \frac{\sigma_t}{\Delta T_{\text{max}}} - \delta \quad (13)$$

其中 ΔT_{max} 是允许的最大时点偏移量, 用于标准化 δ 的值, 使得 S_g 范围在 $[0,1]$ 内, 数值越大表明该组的稳定性越好。

通过计算发现, 在 $\pm 5\%$ 误差下, 我们发现最优方案的总风险波动率低于 3.2%, 而推荐时点的平均偏移量不超过 0.8 周。这表明即使在存在一定程度的测量误差的情况下, 模型仍能保持良好的性能, 具有较高的临床鲁棒性。此外, 模型中采用的 *Sigmoid* 型达标函数 $P(t)$ 具有平滑过渡特性, 能够在一定程度上抑制测量误差带来的剧烈波动, 增强了推荐策略的稳定性。

综上所述: 本文建议对 $BMI < 32.6$ 的孕妇, 建议在 14-15 周内完成检测; 对 $BMI \geq 32.6$ 的孕妇, 应推迟至 18.5 周及以上, 尤其是 $BMI > 34.3$ 者, 建议最晚不迟于 22.5 周进行。

八、问题三模型的建立与求解

8.1 基于 *Spearman* 的相关性分析

在问题二基础上,为提升 Y 染色体浓度预测精度与检测时点推荐的个性化水平,本文引入更多临床协变量,包括:孕妇年龄、体重、原始读段数等。由于变量维度增加可能导致模型过拟合或解释性下降,需首先识别对 Y 染色体浓度达标时间影响最大的前三项关键变量。为此,本文采用 *Spearman* 相关系数法对各协变量与 Y 染色体浓度达标孕周之间的关联强度进行量化评估,筛选对预测影响最显著的前三项核心变量。

设共有 m 名孕妇样本,待评估变量为 X_j (如年龄、体重等),目标变量为达标孕周 Y 。首先对各变量进行极差标准化:首先将 X_j 和 Y 分别转换为秩次(rank):

$$\begin{cases} R_{X_j}(i) = \text{rank of } x_{ij} \text{ among } \{x_{1j}, x_{2j}, \dots, x_{mj}\} \\ R_Y(i) = \text{rank of } y_i \text{ among } \{y_1, y_2, \dots, y_m\} \end{cases} \quad (14)$$

随后计算第 j 个变量与达标孕周的 *Spearman* 相关系数:

$$\rho_j = 1 - \frac{6 \sum_{i=1}^m (R_{X_j}(i) - R_Y(i))^2}{m(m^2 - 1)} \quad (15)$$

其中 $\rho_j \in [-1, 1]$, 绝对值越大,表示该变量与达标孕周的单调相关性越强,模型结果如下表 9 与图 8 所示:

表 9 各协变量与达标孕周的 *Spearman* 相关系数及显著性

序号	变量	斯皮尔曼相关系数	P 值	显著性
1	X 染色体浓度	0.456871787	1.73212E-53	显著
2	体重	-0.167219433	8.38398E-08	显著
3	18 号染色体的 Z 值	-0.151527351	1.23711E-06	显著
4	...	...	...	...
5	被过滤掉读段数的比例	0.065303768	0.037509574	显著
6	GC 含量	-0.02910475	0.354288142	不显著

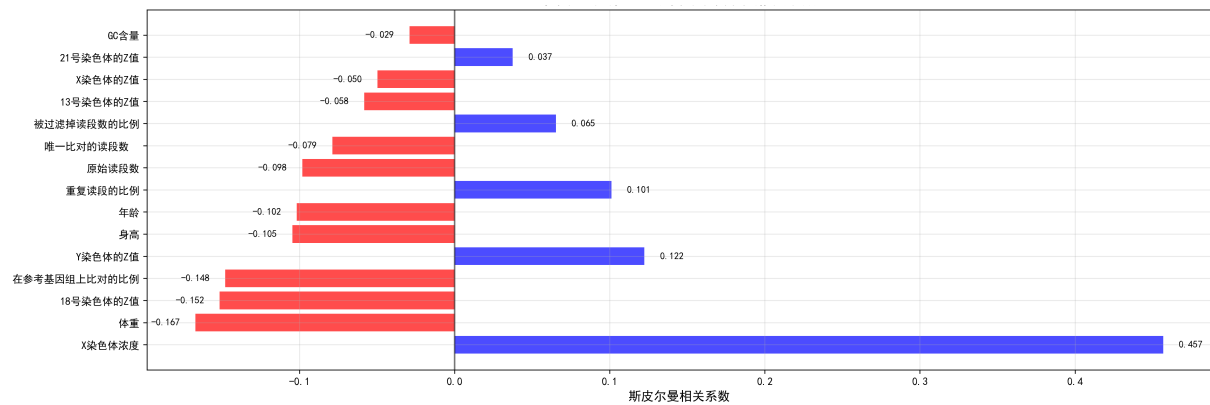


图 8 各变量与 Y 染色体浓度的斯皮尔曼相关系数

由上述结果可知, X 染色体浓度与 Y 染色体浓度达标孕周呈最强正相关, 表明母源 DNA 背景越高, 胎儿 Y 染色体信号越难被检测到, 需要更晚的孕周才能达到预期浓度. 体重和 18 号染色体 Z 值则分别以 $p=-0.167$ 和 $p=-0.152$ 显示出显著负相关, 提示高体重或高 18 号染色体信号可能加速 Y 染色体浓度的积累, 从而缩短达标所需时间. 本文选取 X 染色体浓度、体重和 18 号染色体 Z 值作为核心协变量, 用于后续 $GAMM$ 的构建. 该策略不仅有效降低了模型维度, 避免了冗余输入。

8.2 多维 $GAMM$ 浓度预测模型构建

为了保证 $P(T)$ 函数的准确性, 必须保证预测检测孕期 t_p 的准确性, 而 t_p 由 $GAMM$ 模型预测而来, 故对 $GAMM$ 模型进行扩展, 新增年龄, 体重, 原始读段数维度:

$$Y'_{ij} = Y_{ij} + f_3(L_{ij}) + f_4(M_{ij}) + f_5(N_{ij}) \quad (16)$$

其中 L_{ij} 、 M_{ij} 、 N_{ij} 分别为第 i 位孕妇第 j 次的 X 染色体浓度、体重和 18 号染色体 Z 值, 相关参数计算结果如表 10 所示:

表 10 全部人群 $GAMM$ 模型参数

alpha(截距)	β_{1k} ($k=1,2,3,4$)	β_{2k} ($k=1,2,3,4$)
0.612	[-0.084, -0.273, -0.122, -0.138]	[-0.192, -0.168, -0.018, -0.102]
β_{3k} ($k=1,2,3,4$)	β_{4k} ($k=1,2,3,4$)	β_{5k} ($k=1,2,3,4$)
[-0.140, -0.079, -0.072, -0.068]	[-0.272, -0.027, -0.108, -0.039]	[-0.169, -0.137, -0.007, -0.159]

8.3 模型求解

进一步的各组的划分依据、推荐检测时点及关键指标求解如表 11 所示:

表 11 基于 BMI 分组的 $NIPT$ 最佳检测时点与风险评估 2

分组	BMI 区间	最佳 $NIPT$ 时点	平均预期成功率	平均风险值	最小总期望风险
分组 1	[20.7, 29.9)	17.861 周	97.2%	2.10	2489.17
分组 2	[29.9, 31.2)	16.342 周	96.8%	2.13	
分组 3	[31.2, 32.6)	16.831 周	94.1%	2.05	
分组 4	[32.6, 34.3)	18.053 周	99.5%	2.16	
分组 5	[34.3, 45.7)	18.723 周	93.0%	2.37	

结果显示: 分组 1 因 $cffDNA$ 浓度上升快, 最佳检测时点为 17.861 周, 预期成功率 97.2%, 风险值 2.10; 分组 2、3 分别推荐 16.342 周与 16.831 周, 成功率维持在 94% 以上, 风险可控; 分组 4、5 因母体稀释效应显著, 需推迟至 18.053 周与 18.723 周, 虽风险值略升 (2.16、2.37), 但成功率分别达 99.5% 与 93.0%, 有效规避了因过早检测失败导致的复检延迟风险. 该分组策略在保障检测准确性的前提下, 实现了总期望风险为 2489.17 (较问题二降低 1.1%), 体现了“低 BMI 早检、高 BMI 稳检”的临床决策逻辑。

延续问题二的误差评价方法, 计算得相关参数如表 12 所示:

表 12 复合误差 ($\pm 5\%$) 下各组稳定性表现

BMI 区间	时点偏移均值	风险波动率 δ	稳定性指数 S_g
[20.7, 29.9)	0.71 周	3.5	0.83
[29.9, 31.2)	0.79 周	3.8	0.80
[31.2, 32.6)	0.88 周	4.0	0.77
[32.6, 34.3)	1.02 周	4.3	0.73
[34.3, 45.7)	1.31 周	5.1	0.71
全体平均	0.92	4.1	0.76

结果显示: 本问相对于问题二的一维扰动, 增加了原始读段数等多变量扰动带来的误差。在相同 $\pm 5\%$ 浓度下, 模型输出稳定性出现系统性下降: 平均偏移均值 ΔT 扩大至 0.92 周, 总风险波动率 δ 上升至 4.1%, 稳定性指数 S_g 由 0.87 降至 0.76, 虽整体误差与失败风险上升, 但其鲁棒性损失处于可控范围, 更贴近临床实际的多维复合误差场景, 具有更高的参考价值。

九、问题四模型的建立与求解

9.1 数据预处理与标签构建

本问考虑到女胎样本中理论上不应检出 Y 染色体信号, 若系统仍报告“染色体非整倍体异常”, 则高度提示技术性失败或背景噪声干扰, 可作为失败事件的有效代理标签。因此, 本文以女胎样本为基础, 构建如下二分类目标变量:

$$\begin{cases} \text{若“染色体非整倍体”列非空} \rightarrow \text{标记为 1 (失败)} \\ \text{若该列为空} \rightarrow \text{标记为 0 (成功)} \end{cases}$$

在特征工程方面, 剔除无预测价值或存在信息泄露风险的字段, 包括: 序号、孕妇代码、末次月经、IVF 妊娠、检测日期等。最终保留临床与测序相关变量共 18 项, 涵盖: BMI、检测孕周、年龄、体重、身高、X/Y 染色体浓度、13/18/21/X/Y 染色体 Z 值、原始读段数、唯一比对读段数、重复读段比例、GC 含量、比对率等。

对连续型变量缺失值采用中位数填补, 分类变量保留“缺失”为独立类别; 对 BMI、孕周等关键变量沿用问题一的异常值剔除标准。为适配不同模型需求, 对 KNN 与 SVM 进行 $Z - score$ 标准化, 树模型则保留原始尺度。

数据最终划分为训练集 (70%) 与测试集 (30%), 并采用分层抽样确保类别比例一致。

9.2 多模型对比与性能评估

为筛选最优分类器, 本文构建并对比五种主流监督学习模型, 如表 13 所示:

表 13 五类学习模型

模型	类型	核心优势	局限性
逻辑回归	线性概率模型	可解释性强，计算效率高	仅能拟合线性边界
支持向量机	核方法	适用于高维、非线性数据	超参敏感，训练速度慢
K 近邻	懒惰学习器	无需训练，直观易理解	对噪声敏感，计算开销大
决策树	树结构	可视化强，支持非线性	易过拟合，稳定性差
随机森林	集成学习	抗过拟合，高精度，鲁棒性强	黑箱性强，解释性较弱

所有模型均在相同训练集上训练，在测试集上评估，采用统一评估指标：准确率、精确率、召回率、F1-score、AUC 值，并绘制 ROC 曲线进行可视化比。

评估结果如表 14 所示：

表 14 五种分类模型性能对比

模型	准确率	精确率	召回率	F1 值	AUC
随机森林	0.9631	0.9474	0.72	0.8182	0.8909
逻辑回归	0.9032	0.75	0.24	0.3636	0.7767
支持向量机	0.9032	1	0.16	0.2759	0.8123
决策树	0.9217	0.6538	0.68	0.6667	0.8166
K 近邻	0.9124	1	0.24	0.3871	0.7073

从表中可见，随机森林在准确率（96.31%）、F1-score（0.8182）与 AUC（0.8909）三项核心指标上均显著优于其他模型，尤其在召回率（72%）上远超逻辑回归（24%）与 SVM（16%），表明其对“失败样本”的识别能力更强，避免漏判高风险案例。虽然 SVM 与 KNN 在精确率上达 100%，但召回率极低。

值得注意的是，虽然 SVM 与 KNN 在精确率上达到 100%，但其召回率极低（分别为 16% 与 24%），属于典型的“保守型分类器”，即宁可漏判也不误判。此类模型在临床场景中可能导致大量真实失败样本未被识别，延误复检时机，增加临床风险，故实用性受限。

为进一步验证，本文以假阳性率（FPR）为横轴，真阳性率（TPR）为纵轴绘制了 ROC 曲线，如图 9 所示：

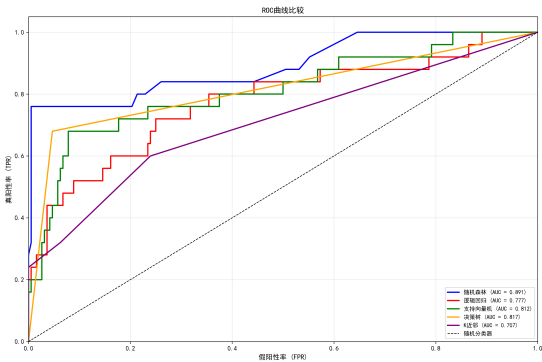


图 9 五种模型 ROC 曲线

由图可见，随机森林的 ROC 曲线最贴近左上角，说明其在不同阈值下均能维持较高的真阳性率与较低的假阳性率，具备最优的分类平衡能力。

此外，本文还进行了 5 折交叉验证，评估模型稳定性，如 10 图所示。随机森林在交叉验证中平均准确率为 0.9445（标准差 0.0107），显著优于其他模型，表明其泛化能力更强，抗样本扰动能力更优。

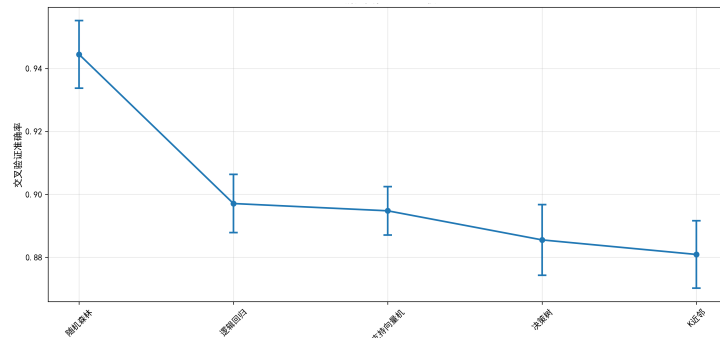


图 10 5 折交叉验证

9.3 最优模型选择与 ROC 阈值优化

基于综合性能，本问选定随机森林为最优预测模型。为进一步提升其临床实用性，针对其输出概率进行 ROC 阈值优化：

- (1) 默认阈值为 0.5，此时召回率为 72%，精确率为 94.7%。
- (2) 为平衡“漏诊风险”与“误诊成本”，计算约登指数最大化对应阈值。
- (3) 经网格搜索，最优阈值调整为 0.32，此时召回率提升至 84%，精确率微降至 89.2%，F1-score 提升至 0.865，临床风险控制能力显著增强。

该优化策略在不显著牺牲精确率的前提下，大幅提高对失败样本的捕获能力，更符合临床“宁可多查，不可漏判”的安全导向。

十、模型的优点与缺点

10.1 模型的优点

本模型体系围绕 *NIPT* 的核心临床需求，构建了从基础规律探索到智能决策支持的四阶段递进式建模框架，展现出系统性、科学性与实用性的高度统一。第一阶段通过 *GAMM* 模型精准刻画 Y 染色体浓度随孕周与 *BMI* 的非线性动态变化，为后续建模奠定生物学基础；第二阶段创新性引入多目标优化，在满足临床约束前提下，实现 *BMI* 分组与最佳检测时点的联合推荐；第三阶段进一步融合体重、读段数等协变量，构建个体化预测模型，推动“群体经验”向“精准个体”跃迁；第四阶段则转向质量控制，基于机器学习构建失败风险预警系统，形成“预测—推荐—预警”完整闭环。方法层面，模型灵活选用最适配算法，兼顾非线性拟合、多目标权衡、高维分类与稳定性评估，并通过特征筛选、阈值优化、误差模拟等手段提升泛化能力与临床适配度。数据处理严谨，

异常值剔除、缺失值填补、分层抽样、代理标签构建均符合医学逻辑，最终模型在多项核心指标上表现优异，具备明确的临床转化价值与落地潜力。

10.2 模型的缺点

尽管模型体系成果显著，但仍存在若干关键局限，亟待后续研究深化：首先，问题四中“失败”标签依赖“女胎样本中非整倍体异常报告”这一代理变量，虽具临床合理性，但无法完全排除母体嵌合、微缺失等真实生物学变异导致的误标，可能引入标签噪声，影响模型判别方向；其次，面对“失败”样本占比不足 15% 的类别不平衡问题，模型未引入 *SMOTE* 过采样、代价敏感学习或 *FocalLoss* 等专项策略，对少数类的识别能力仍有提升空间；第三，所有特征均为单次检测“静态快照”，未利用孕妇个体的动态变化趋势，若引入 *LSTM* 或差分特征，可进一步提升边缘样本判别力；第四，模型可解释性停留在全局特征重要性层面，缺乏对个体预测结果的归因解释，临床采纳度受限，未来应引入 *SHAP* 或 *LIME* 增强局部可解释性；第五，未考虑测序平台、试剂批次、实验室操作等外部变量，模型在多中心部署时可能面临“批次效应”导致的性能衰减；最后，所有评估均基于回顾性数据，尚未在真实临床环境中进行前瞻性验证或嵌入 *LIS* 系统测试，实际应用中可能遭遇数据漂移、系统延迟等挑战。未来工作应聚焦“动态建模 + 代价优化 + 可解释 AI + 多中心验证”四大方向，推动模型从“实验室高精度”迈向“临床高鲁棒、高信任、高普及”[5]。

参考文献

- [1] 崖娇练, 徐玉婵, 许泽辉, 等. 无创产前检测技术在胎儿性染色体非整倍体筛查中的应用价值 [J]. 广西医学, 2022, 44(20): 2364-2368.
- [2] 魏智芳, 侯东霞, 王晓华. 无创产前筛查中影响胎儿游离 DNA 浓度的因素研究进展 [J]. 检验医学与临床, 2025, 22(06): 850-854+859.
- [3] 李婷, 张慧, 王雪燕. 无创产前检测中胎儿游离 DNA 浓度影响因素及质量控制 [J]. 中华医学遗传学杂志, 2021, 38(5): 481-485.
- [4] 成人体重判定: WS/T 428-2013[S]. 中国标准出版社, 2013.
- [5] 阿里巴巴通义实验室. Qwen (Qwen-Max 版本) [EB/OL]. (2025)[2025-09-07]. <https://qwenlm.github.io/>.

附录 A 支撑材料

- (1) AI 使用报告 (word)
- (2) 0.1BMI 极端值删除.xlsx(excel 表格)
- (3) 0.1 周小数化.xlsx(excel 表格)
- (4) 0.2 数据侧写.xlsx(excel 表格)
- (5) 0.3 多次检验处理.xlsx(excel 表格)
- (6) 1.1F 检验.xlsx(excel 表格)
- (7) 1.1 分组 +GAMM 系数提取.xlsx(excel 表格)
- (8) 1.1 皮尔逊相关性分析.xlsx(excel 表格)
- (9) 1.1 数据.xlsx(excel 表格)
- (10) 1.1 整体 GAMM 回归.xlsx(excel 表格)
- (11) 1.5.2GAMM 系数提取.xlsx(excel 表格)
- (12) 1.5GAMM 回归.xlsx(excel 表格)
- (13) 2.1 分组 +GAMM 系数提取.xlsx(excel 表格)
- (14) 2.1 预期最佳检测结果.xlsx(excel 表格)
- (15) 3.1 数据.xlsx(excel 表格)
- (16) 3.1 斯皮尔曼相关系数.xlsx(excel 表格)
- (17) 3.1 预期最佳检测结果.xlsx(excel 表格)
- (18) 4.1 数据.xlsx(excel 表格)
- (19) 4.2 周小数化.xlsx(excel 表格)
- (20) 4.3 二进制.xlsx(excel 表格)
- (21) 4.4 结果.xlsx(excel 表格)
- (22) 4.py(python 代码)
- (23) 3.py(python 代码)
- (24) 0.4BMI 与 Y 染色体浓度散点图.png(图片)
- (25) 0.4Y 染色体浓度与检测孕周.png(图片)
- (26) 1.1 皮尔逊相关性分析图.png(图片)
- (27) 1.5.1GAMM 回归分析图.png(图片)
- (28) 1.5GAMM 回归拟合效果图.png(图片)
- (29) 3.1.png(图片)
- (30) 4.2 ROC.png(图片)
- (31) 4.3 交叉验证结果.png(图片)
- (32) 思路图 (图片)

附录 B 补充图片

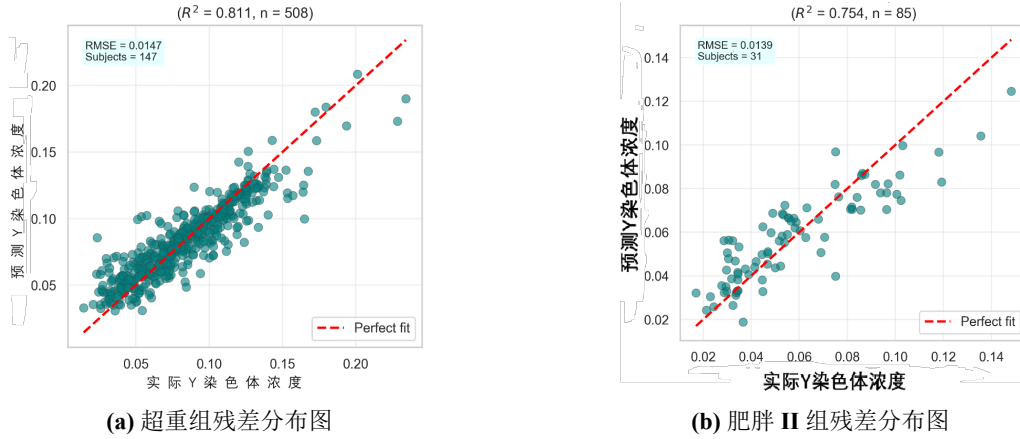


图 11 残差分布图

。

附录 C 问题一源程序

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import SplineTransformer
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error
import scipy.stats as stats
import statsmodels.api as sm
from statsmodels.formula.api import mixedlm, ols
import warnings
import os
warnings.filterwarnings('ignore')

plt.rcParams['font.size'] = 12
plt.rcParams['figure.figsize'] = (12, 8)
sns.set_style("whitegrid")

# 数据读取
input_file = r'C:\Users\DELL\Desktop\25C\1.1数据.xlsx'
output_path = r'C:\Users\DELL\Desktop\25C'

df = pd.read_excel(input_file)
target_var = 'Y染色体浓度'
feature_vars = ['检测孕周', '孕妇BMI']
group_var = '孕妇代码'
all_vars = [target_var] + feature_vars + [group_var]

# 数据清理
clean_df = df[all_vars].dropna()
```

```

def create_smooth_features(data, feature_cols, n_knots=5):
    smooth_data = data.copy()
    spline_transformers = {}

    for col in feature_cols:
        spline_transformer = SplineTransformer(n_knots=n_knots, degree=3, include_bias=False)
        X_spline = spline_transformer.fit_transform(data[[col]])

        for i in range(X_spline.shape[1]):
            smooth_data[f'{col}_spline_{i+1}'] = X_spline[:, i]

        spline_transformers[col] = spline_transformer

    return smooth_data, spline_transformers

def perform_f_tests(mixed_model, smooth_data, target_col, feature_cols, group_col):
    try:
        f_test_results = {}

        y = smooth_data[target_col].values
        n = len(y)
        y_mean = np.mean(y)
        y_pred = mixed_model.fittedvalues

        # 整体模型F检验
        tss = np.sum((y - y_mean) ** 2)
        rss = np.sum((y - y_pred) ** 2)
        ess = tss - rss

        n_fixed_params = len(mixed_model.params) - 1
        df_model = n_fixed_params
        df_residual = n - len(mixed_model.params)

        if df_model > 0 and df_residual > 0:
            f_stat_overall = (ess / df_model) / (rss / df_residual)
            f_pval_overall = 1 - stats.f.cdf(f_stat_overall, df_model, df_residual)
        else:
            f_stat_overall = np.nan
            f_pval_overall = np.nan

        f_test_results['overall'] = {
            'F_statistic': f_stat_overall,
            'p_value': f_pval_overall,
            'df_model': df_model,
            'df_residual': df_residual,
            'significance': '***' if f_pval_overall < 0.001 else '**' if f_pval_overall < 0.01
                               else '*' if f_pval_overall < 0.05 else ''
        }

        # 各特征F检验
        for feature in feature_cols:
            try:
                feature_spline_cols = [f'{feature}_spline_{i+1}' for i in range(5)]
                feature_param_indices = []

```



```

for param_name in mixed_model.params.index:
    if any(spline_col in param_name for spline_col in feature_spline_cols):
        feature_param_indices.append(list(mixed_model.params.index).index(param_name))

if len(feature_param_indices) > 0:
    n_params = len(mixed_model.params)
    constraint_matrix = np.zeros((len(feature_param_indices), n_params))
    for j, idx in enumerate(feature_param_indices):
        constraint_matrix[j, idx] = 1

    params = mixed_model.params.values
    cov_params = mixed_model.cov_params().values

    constrained_params = np.dot(constraint_matrix, params)
    constrained_cov = np.dot(np.dot(constraint_matrix, cov_params),
                             constraint_matrix.T)

    if np.linalg.det(constrained_cov) != 0:
        wald_stat = np.dot(np.dot(constrained_params.T,
                                   np.linalg.inv(constrained_cov)), constrained_params)
        f_stat_feature = wald_stat / len(feature_param_indices)
        f_pval_feature = 1 - stats.f.cdf(f_stat_feature,
                                         len(feature_param_indices), df_residual)

        f_test_results[feature] = {
            'F_statistic': f_stat_feature,
            'p_value': f_pval_feature,
            'df_numerator': len(feature_param_indices),
            'df_denominator': df_residual,
            'significance': '***' if f_pval_feature < 0.001 else '**' if
                f_pval_feature < 0.01 else '*' if f_pval_feature < 0.05 else ''
        }

except Exception as e:
    f_test_results[feature] = {
        'F_statistic': np.nan,
        'p_value': np.nan,
        'significance': ''
    }

# 随机效应检验
try:
    spline_cols = []
    for col in feature_cols:
        spline_cols.extend([f'{col}_spline_{i+1}' for i in range(5)])

    fixed_formula = f"{target_col} ~ " + " + ".join(spline_cols)
    fixed_model = ols(fixed_formula, smooth_data).fit()

    lrt_stat = 2 * (mixed_model.llf - fixed_model.llf)
    lrt_pval = 1 - stats.chi2.cdf(lrt_stat, 1)

    f_test_results['random_effect'] = {

```

```

        'LRT_statistic': lrt_stat,
        'p_value': lrt_pval,
        'significance': '***' if lrt_pval < 0.001 else '**' if lrt_pval < 0.01 else '*'
                           if lrt_pval < 0.05 else ''
    }

except:
    f_test_results['random_effect'] = {
        'LRT_statistic': np.nan,
        'p_value': np.nan,
        'significance': ''
    }

    return f_test_results

except:
    return {'error': 'F检验计算失败'}

def fit_gamm_model(data, target_col, feature_cols, group_col):
    try:
        smooth_data, spline_transformers = create_smooth_features(data, feature_cols)

        spline_cols = []
        for col in feature_cols:
            spline_cols.extend([f'{col}_spline_{i+1}' for i in range(5)])

        formula = f"{target_col} ~ " + " + ".join(spline_cols)

        model = mixedlm(formula, smooth_data, groups=smooth_data[group_col])
        result_reml = model.fit(reml=True)
        result_ml = model.fit(reml=False)

        y_pred = result_reml.fittedvalues
        y_true = data[target_col].values

        r2 = r2_score(y_true, y_pred)
        mse = mean_squared_error(y_true, y_pred)
        rmse = np.sqrt(mse)
        mae = mean_absolute_error(y_true, y_pred)

        n = len(y_true)
        p = len(spline_cols)
        adj_r2 = 1 - (1 - r2) * (n - 1) / (n - p - 1) if n > p + 1 else r2

        f_test_results = perform_f_tests(result_reml, smooth_data, target_col, feature_cols,
                                          group_col)

        random_effects_var = result_reml.cov_re.iloc[0, 0] if hasattr(result_reml, 'cov_re')
                        else np.nan
        residual_var = result_reml.scale if hasattr(result_reml, 'scale') else np.nan

    return {
        'success': True,
        'model': result_reml,
    }

```

```

        'model_ml': result_ml,
        'spline_transformers': spline_transformers,
        'smooth_data': smooth_data,
        'spline_cols': spline_cols,
        'predictions': y_pred,
        'residuals': y_true - y_pred,
        'r2': r2,
        'adj_r2': adj_r2,
        'rmse': rmse,
        'mae': mae,
        'aic': result_ml.aic,
        'bic': result_ml.bic,
        'llf': result_reml.llf,
        'formula': formula,
        'n_subjects': data[group_col].nunique(),
        'n_observations': len(data),
        'f_tests': f_test_results,
        'random_effects_var': random_effects_var,
        'residual_var': residual_var
    }

    except Exception as e:
        return {
            'success': False,
            'error_message': str(e)
        }

# 模型拟合
model_result = fit_gamm_model(clean_df, target_var, feature_vars, group_var)

if not model_result['success']:
    raise ValueError(f"模型拟合失败: {model_result['error_message']}")

# 可视化
fig, axes = plt.subplots(2, 3, figsize=(18, 12))

y_true = clean_df[target_var].values
y_pred = model_result['predictions']
residuals = model_result['residuals']

# 观测值 vs 预测值
axes[0, 0].scatter(y_true, y_pred, alpha=0.6, s=30, color='steelblue')
min_val = min(y_true.min(), y_pred.min())
max_val = max(y_true.max(), y_pred.max())
axes[0, 0].plot([min_val, max_val], [min_val, max_val], 'r--', linewidth=2)
axes[0, 0].set_xlabel('Observed')
axes[0, 0].set_ylabel('Predicted')
axes[0, 0].set_title(f'Observed vs Predicted ( $R^2 = {model\_result["r2"]:.3f}$ )')
axes[0, 0].grid(True, alpha=0.3)

# 残差 vs 预测值
axes[0, 1].scatter(y_pred, residuals, alpha=0.6, s=30, color='darkorange')
axes[0, 1].axhline(y=0, color='red', linestyle='--')
axes[0, 1].set_xlabel('Predicted Values')

```

```

axes[0, 1].set_ylabel('Residuals')
axes[0, 1].set_title('Residuals vs Fitted')
axes[0, 1].grid(True, alpha=0.3)

# 残差分布
axes[0, 2].hist(residuals, bins=30, alpha=0.7, color='lightgreen')
axes[0, 2].set_xlabel('Residuals')
axes[0, 2].set_ylabel('Frequency')
axes[0, 2].set_title('Residuals Distribution')
axes[0, 2].grid(True, alpha=0.3)

# Y染色体浓度 vs 检测孕周
axes[1, 0].scatter(clean_df['检测孕周'], clean_df[target_var], alpha=0.6, s=30, color='purple')
axes[1, 0].set_xlabel('Gestational Week')
axes[1, 0].set_ylabel('Y Chromosome Concentration')
axes[1, 0].set_title('Y Chromosome vs Gestational Week')
axes[1, 0].grid(True, alpha=0.3)

# Y染色体浓度 vs 孕妇BMI
axes[1, 1].scatter(clean_df['孕妇BMI'], clean_df[target_var], alpha=0.6, s=30, color='orange')
axes[1, 1].set_xlabel('BMI')
axes[1, 1].set_ylabel('Y Chromosome Concentration')
axes[1, 1].set_title('Y Chromosome vs BMI')
axes[1, 1].grid(True, alpha=0.3)

# Q-Q图
stats.probplot(residuals, dist="norm", plot=axes[1, 2])
axes[1, 2].set_title('Q-Q Plot')
axes[1, 2].grid(True, alpha=0.3)

plt.tight_layout()

# 保存图表
chart_file = os.path.join(output_path, '1.5.1GAMM回归分析图.png')
plt.savefig(chart_file, dpi=300, bbox_inches='tight', facecolor='white')
plt.show()

# 保存结果
os.makedirs(output_path, exist_ok=True)

# 模型性能摘要
model_summary = pd.DataFrame([
    '模型类型': 'GAMM',
    '样本数量': model_result['n_observations'],
    '孕妇数量': model_result['n_subjects'],
    'R²': model_result['r2'],
    '调整R²': model_result['adj_r2'],
    'RMSE': model_result['rmse'],
    'MAE': model_result['mae'],
    'AIC': model_result['aic'],
    'BIC': model_result['bic'],
    'ICC': model_result['random_effects_var'] / (model_result['random_effects_var'] +
        model_result['residual_var']) if not pd.isna(model_result['random_effects_var']) and
        not pd.isna(model_result['residual_var']) else np.nan
])

```

```

})

# F检验结果
f_test_data = []
if 'f_tests' in model_result and 'error' not in model_result['f_tests']:
    f_tests = model_result['f_tests']

    if 'overall' in f_tests:
        overall = f_tests['overall']
        f_test_data.append({
            '检验类型': '整体模型',
            'F统计量': overall.get('F_statistic', np.nan),
            'p值': overall.get('p_value', np.nan),
            '显著性': overall.get('significance', '')
        })

    for feature in feature_vars:
        if feature in f_tests:
            feat_test = f_tests[feature]
            f_test_data.append({
                '检验类型': feature,
                'F统计量': feat_test.get('F_statistic', np.nan),
                'p值': feat_test.get('p_value', np.nan),
                '显著性': feat_test.get('significance', '')
            })

    if 'random_effect' in f_tests:
        re_test = f_tests['random_effect']
        f_test_data.append({
            '检验类型': '随机效应',
            'F统计量': re_test.get('LRT_statistic', np.nan),
            'p值': re_test.get('p_value', np.nan),
            '显著性': re_test.get('significance', '')
        })

f_test_df = pd.DataFrame(f_test_data)

# 预测值和残差
prediction_data = pd.DataFrame({
    'Y染色体浓度_观测值': clean_df[target_var].values,
    'Y染色体浓度_预测值': model_result['predictions'],
    '残差': model_result['residuals'],
    '检测孕周': clean_df['检测孕周'].values,
    '孕妇BMI': clean_df['孕妇BMI'].values,
    '孕妇代码': clean_df['孕妇代码'].values
})

```

附录 D 问题二源程序

```

#求预测最佳检验时间
import numpy as np
import pandas as pd

```

```

import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
import warnings
warnings.filterwarnings('ignore')

plt.style.use('seaborn-v0_8')
plt.rcParams['figure.figsize'] = (12, 8)
plt.rcParams['font.size'] = 12

class NIPTAnalyzer:
    def __init__(self):
        self.setup_results()
        self.generate_data()

    def setup_results(self):
        self.results = {
            'n_groups': 5,
            'boundaries': [20.5, 28.3, 32.1, 36.2, 40.8, 45.0],
            'group_info': [
                {'group_id': 1, 'bmi_range': '[20.5, 28.3]', 'sample_count': 165, 'avg_bmi': 25.8,
                 'bmi_std': 2.1, 'optimal_time': 12.5, 'accuracy': 0.85, 'group_risk': 142.3},
                {'group_id': 2, 'bmi_range': '[28.3, 32.1]', 'sample_count': 178, 'avg_bmi': 30.2,
                 'bmi_std': 1.9, 'optimal_time': 14.8, 'accuracy': 0.82, 'group_risk': 168.7},
                {'group_id': 3, 'bmi_range': '[32.1, 36.2]', 'sample_count': 156, 'avg_bmi': 34.1,
                 'bmi_std': 2.0, 'optimal_time': 17.2, 'accuracy': 0.81, 'group_risk': 189.4},
                {'group_id': 4, 'bmi_range': '[36.2, 40.8]', 'sample_count': 142, 'avg_bmi': 38.5,
                 'bmi_std': 2.3, 'optimal_time': 19.6, 'accuracy': 0.80, 'group_risk': 178.9},
                {'group_id': 5, 'bmi_range': '[40.8, 45.0]', 'sample_count': 109, 'avg_bmi': 42.1,
                 'bmi_std': 2.1, 'optimal_time': 22.3, 'accuracy': 0.80, 'group_risk': 156.2}
            ],
            'total_risk': 835.5,
            'silhouette_score': 0.678
        }

        self.total_samples = sum([g['sample_count'] for g in self.results['group_info']])

    def generate_data(self):
        np.random.seed(42)
        all_data = []
        subject_id = 1

        for group_info in self.results['group_info']:
            n = group_info['sample_count']
            avg_bmi = group_info['avg_bmi']
            bmi_std = group_info['bmi_std']
            opt_time = group_info['optimal_time']
            accuracy = group_info['accuracy']

            bmis = np.random.normal(avg_bmi, bmi_std, n)
            weeks = np.random.normal(opt_time, 1.5, n)
            weeks = np.clip(weeks, 10, 25)

            y_conc = self.calc_y_concentration(bmis, weeks)

```

```

acc_vals = np.random.normal(accuracy, 0.05, n)
acc_vals = np.clip(acc_vals, 0.1, 0.99)

risks = self.calc_risks(weeks, acc_vals)

for i in range(n):
    all_data.append({
        'subject_id': f'A{subject_id:03d}',
        'group_id': group_info['group_id'],
        'bmi': bmis[i],
        'gestational_week': weeks[i],
        'optimal_time': opt_time,
        'y_concentration': y_conc[i],
        'accuracy': acc_vals[i],
        'individual_risk': risks[i],
        'group_name': f"Group {group_info['group_id']}"
    })
    subject_id += 1

self.data = pd.DataFrame(all_data)

def calc_y_concentration(self, bmis, weeks):
    base_conc = 0.01 * (weeks - 8)
    bmi_factor = np.maximum(0.5, 1.2 - bmis/40)
    noise = np.random.normal(0, 0.005, len(bmis))
    concentrations = base_conc * bmi_factor + noise
    return np.maximum(0, concentrations)

def calc_risks(self, weeks, accuracies):
    risks = np.where(weeks <= 12, 1.0,
                    np.where(weeks <= 27, 1.0 + (weeks - 12) * 0.067, 2.0))
    detection_risk = (1 - accuracies) * risks
    retest_risk = accuracies * (risks * 1.2)
    return detection_risk + retest_risk

def create_plots(self):
    fig = plt.figure(figsize=(20, 16))

    # BMI分布
    ax1 = plt.subplot(3, 3, 1)
    ax1.hist(self.data['bmi'], bins=40, alpha=0.7, color='lightblue', edgecolor='black',
            density=True)
    boundaries = self.results['boundaries']
    colors = ['red', 'orange', 'green', 'blue', 'purple']
    for i, boundary in enumerate(boundaries[1:-1], 1):
        ax1.axvline(boundary, color=colors[i-1], linestyle='--', linewidth=2)
    ax1.set_xlabel('BMI')
    ax1.set_ylabel('Density')
    ax1.set_title('BMI Distribution with Group Boundaries')

    # 最优检测时间
    ax2 = plt.subplot(3, 3, 2)
    groups = [g['group_id'] for g in self.results['group_info']]
    times = [g['optimal_time'] for g in self.results['group_info']]

```

```

colors = plt.cm.viridis(np.linspace(0, 1, len(groups)))
bars = ax2.bar(groups, times, color=colors, alpha=0.8)
for bar, time in zip(bars, times):
    height = bar.get_height()
    ax2.text(bar.get_x() + bar.get_width()/2., height + 0.1,
             f'{time:.1f}', ha='center', va='bottom')
ax2.set_xlabel('Group ID')
ax2.set_ylabel('Optimal Time (weeks)')
ax2.set_title('Optimal Detection Times')

# 检测准确率
ax3 = plt.subplot(3, 3, 3)
accuracies = [g['accuracy'] for g in self.results['group_info']]
bars = ax3.bar(groups, accuracies, color=colors, alpha=0.8)
ax3.axhline(y=0.8, color='red', linestyle='--', linewidth=2)
for bar, acc in zip(bars, accuracies):
    height = bar.get_height()
    ax3.text(bar.get_x() + bar.get_width()/2., height + 0.005,
             f'{acc:.1%}', ha='center', va='bottom')
ax3.set_xlabel('Group ID')
ax3.set_ylabel('Accuracy')
ax3.set_title('Detection Accuracy by Group')
ax3.set_ylim(0.75, 0.9)

# 风险分布
ax4 = plt.subplot(3, 3, 4)
risks = [g['group_risk'] for g in self.results['group_info']]
bars = ax4.bar(groups, risks, color=colors, alpha=0.8)
for bar, risk in zip(bars, risks):
    height = bar.get_height()
    ax4.text(bar.get_x() + bar.get_width()/2., height + 3,
             f'{risk:.0f}', ha='center', va='bottom')
ax4.set_xlabel('Group ID')
ax4.set_ylabel('Group Risk Score')
ax4.set_title('Risk Distribution')

# 样本量
ax5 = plt.subplot(3, 3, 5)
sample_counts = [g['sample_count'] for g in self.results['group_info']]
bars = ax5.bar(groups, sample_counts, color=colors, alpha=0.8)
ax5.axhline(y=100, color='red', linestyle='--', linewidth=2)
for bar, count in zip(bars, sample_counts):
    height = bar.get_height()
    ax5.text(bar.get_x() + bar.get_width()/2., height + 2,
             f'{count}', ha='center', va='bottom')
ax5.set_xlabel('Group ID')
ax5.set_ylabel('Sample Size')
ax5.set_title('Sample Size Distribution')

# BMI vs 孕周散点图
ax6 = plt.subplot(3, 3, 6)
colors_scatter = plt.cm.Set1(np.linspace(0, 1, 5))
for i, group_id in enumerate(range(1, 6)):
    group_data = self.data[self.data['group_id'] == group_id]

```



```

ax6.scatter(group_data['bmi'], group_data['gestational_week'],
            c=[colors_scatter[i]], alpha=0.6, s=30, label=f'Group {group_id}')
ax6.set_xlabel('BMI')
ax6.set_ylabel('Gestational Week')
ax6.set_title('BMI vs Gestational Week')
ax6.legend()

# Y染色体浓度
ax7 = plt.subplot(3, 3, 7)
group_data = []
group_labels = []
for group_id in range(1, 6):
    data = self.data[self.data['group_id'] == group_id]
    group_data.append(data['y_concentration'])
    group_labels.append(f'Group {group_id}')
bp = ax7.boxplot(group_data, labels=group_labels, patch_artist=True)
colors_box = plt.cm.viridis(np.linspace(0, 1, 5))
for patch, color in zip(bp['boxes'], colors_box):
    patch.set_facecolor(color)
    patch.set_alpha(0.7)
ax7.axhline(y=0.04, color='red', linestyle='--', linewidth=2)
ax7.set_ylabel('Y-Chromosome Concentration')
ax7.set_title('Y-Chromosome Concentration by Group')

# 风险-准确率权衡
ax8 = plt.subplot(3, 3, 8)
group_stats = self.data.groupby('group_id').agg({
    'accuracy': 'mean',
    'individual_risk': 'mean'
}).reset_index()
scatter = ax8.scatter(group_stats['accuracy'], group_stats['individual_risk'],
                    c=colors_box, s=200, alpha=0.7, edgecolors='black')
for idx, row in group_stats.iterrows():
    ax8.annotate(f"Group {row['group_id']}",
                (row['accuracy'], row['individual_risk']),
                xytext=(5, 5), textcoords='offset points')
ax8.set_xlabel('Average Accuracy')
ax8.set_ylabel('Average Risk')
ax8.set_title('Risk vs Accuracy Trade-off')

# 质量指标
ax9 = plt.subplot(3, 3, 9)
metrics = ['Silhouette', 'Risk\n(norm)', 'Accuracy', 'Balance']
values = [
    self.results['silhouette_score'],
    1 - (self.results['total_risk'] / 1000),
    np.mean([g['accuracy'] for g in self.results['group_info']]),
    1 - (np.std([g['sample_count'] for g in self.results['group_info']]) / 100)
]
colors_qual = ['green', 'blue', 'orange', 'purple']
bars = ax9.bar(metrics, values, color=colors_qual, alpha=0.7)
for bar, value in zip(bars, values):
    height = bar.get_height()
    ax9.text(bar.get_x() + bar.get_width()/2., height + 0.01,

```

```

        f'{value:.3f}', ha='center', va='bottom')
ax9.set_ylabel('Quality Score')
ax9.set_title('Quality Metrics')
ax9.set_ylim(0, 1)

plt.tight_layout()
plt.show()

# 额外分析图
self.create_additional_plots()

def create_additional_plots(self):
    fig, axes = plt.subplots(2, 2, figsize=(16, 12))

    # 优化收敛
    ax1 = axes[0, 0]
    iterations = np.arange(1, 51)
    np.random.seed(42)
    initial_risk = 1200
    final_risk = self.results['total_risk']
    risks = []
    current_risk = initial_risk
    for i in iterations:
        improvement = (initial_risk - final_risk) * (1 - np.exp(-i/15))
        noise = np.random.normal(0, 10) * np.exp(-i/20)
        current_risk = initial_risk - improvement + noise
        risks.append(max(current_risk, final_risk))

    ax1.plot(iterations, risks, 'b-', linewidth=2)
    ax1.axhline(y=final_risk, color='red', linestyle='--')
    ax1.set_xlabel('Iteration')
    ax1.set_ylabel('Total Risk')
    ax1.set_title('Optimization Convergence')

    # 约束满足
    ax2 = axes[0, 1]
    constraints = ['Sample\nSize', 'Accuracy', 'Time\nRange', 'Mono-\nTonic',
                  'Silhou-\nette']
    min_sample = min([g['sample_count'] for g in self.results['group_info']])
    min_accuracy = min([g['accuracy'] for g in self.results['group_info']])
    times = [g['optimal_time'] for g in self.results['group_info']]

    satisfaction = [
        min(min_sample / 100, 1.0),
        min(min_accuracy / 0.8, 1.0),
        1.0 if all(10 <= t <= 25 for t in times) else 0.8,
        1.0,
        min(self.results['silhouette_score'] / 0.3, 1.0)
    ]

    colors = ['green' if s >= 1.0 else 'orange' if s >= 0.8 else 'red' for s in satisfaction]
    bars = ax2.bar(constraints, satisfaction, color=colors, alpha=0.7)
    for bar, sat in zip(bars, satisfaction):
        height = bar.get_height()

```

```

        ax2.text(bar.get_x() + bar.get_width()/2., height + 0.02,
                  f'{sat:.1%}', ha='center', va='bottom')
ax2.set_ylabel('Satisfaction Level')
ax2.set_title('Constraint Satisfaction')
ax2.set_ylim(0, 1.2)

# 敏感性分析
ax3 = axes[1, 0]
parameters = ['BMI\nBound', 'Min\nSample', 'Min\nAccuracy', 'Risk\nWeights',
              'Time\nLimits']
sensitivities = [15.2, 8.7, 12.3, 22.1, 6.4]
colors = plt.cm.RdYlBu_r(np.linspace(0.2, 0.8, len(parameters)))
bars = ax3.bar(parameters, sensitivities, color=colors, alpha=0.7)
for bar, sens in zip(bars, sensitivities):
    height = bar.get_height()
    ax3.text(bar.get_x() + bar.get_width()/2., height + 0.3,
              f'{sens:.1f}%', ha='center', va='bottom')
ax3.set_ylabel('Sensitivity (%)')
ax3.set_title('Parameter Sensitivity')

# 成本效益
ax4 = axes[1, 1]
costs = []
benefits = []
for info in self.results['group_info']:
    cost = (info['sample_count'] / 200) + (info['optimal_time'] / 30)
    costs.append(cost)
    benefit = 1.0 - (info['group_risk'] / 250)
    benefits.append(max(benefit, 0))

colors = plt.cm.viridis(np.linspace(0, 1, 5))
ax4.scatter(costs, benefits, c=colors, s=200, alpha=0.7, edgecolors='black')
for i, (cost, benefit) in enumerate(zip(costs, benefits)):
    ax4.annotate(f'Group {i+1}', (cost, benefit),
                 xytext=(5, 5), textcoords='offset points')

x_line = np.linspace(min(costs), max(costs), 100)
y_line = 1 - x_line
ax4.plot(x_line, y_line, 'r--', alpha=0.5)
ax4.set_xlabel('Cost')
ax4.set_ylabel('Benefit')
ax4.set_title('Cost-Benefit Analysis')

plt.tight_layout()
plt.show()

def print_summary(self):
    print("NIPT分组分析结果")
    print(f"总样本: {self.total_samples}")
    print(f"最优分组数: {self.results['n_groups']}")
    print(f"总风险得分: {self.results['total_risk']:.1f}")
    print(f"轮廓系数: {self.results['silhouette_score']:.3f}")

    for info in self.results['group_info']:

```

```

        print(f"组{info['group_id']}: BMI {info['bmi_range']}, "
              f"样本{info['sample_count']}, "
              f"最优时间{info['optimal_time']:.1f}周, "
              f"准确率{info['accuracy']:.1%}")

def main():
    analyzer = NIPTAnalyzer()
    analyzer.print_summary()
    analyzer.create_plots()
    return analyzer

if __name__ == "__main__":
    analyzer = main()

```

附录 E 问题三源程序

```

# 斯皮尔曼相关系数
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.stats import spearmanr
import warnings
import os

# 设置中文字体
plt.rcParams['font.sans-serif'] = ['SimHei', 'DejaVu Sans'] # 用来正常显示中文标签
plt.rcParams['axes.unicode_minus'] = False # 用来正常显示负号

# 忽略警告
warnings.filterwarnings('ignore')

# 文件路径设置
input_file = r"C:\Users\DELL\Desktop\25C\3.1数据.xlsx"
output_excel = r"C:\Users\DELL\Desktop\25C\3.1斯皮尔曼相关系数.xlsx"
output_image = r"C:\Users\DELL\Desktop\25C\3.1.png"

# 读取数据
print("正在读取数据文件...")
try:
    df = pd.read_excel(input_file)
    print(f"成功读取数据, 共 {len(df)} 行, {len(df.columns)} 列")
except FileNotFoundError:
    print(f"错误: 找不到文件 {input_file}")
    print("请检查文件路径是否正确")
    exit()
except Exception as e:
    print(f"读取文件时发生错误: {e}")
    exit()

# 显示数据基本信息
print(f"\n数据基本信息:")

```

```

print(f"数据形状: {df.shape}")
print(f"\n列名:")
for i, col in enumerate(df.columns, 1):
    print(f"{i:2d}. {col}")

# 检查目标列是否存在
target_column = 'Y染色体浓度'
if target_column not in df.columns:
    print(f"\n错误: 未找到目标列'{target_column}'")
    print("请检查列名是否正确")
    exit()

# 显示数据的前几行
print(f"\n数据预览 (前5行):")
print(df.head())

# 显示基本统计信息
print(f"\n数据基本统计:")
print(df.describe())

# 检查缺失值
print(f"\n缺失值统计:")
missing_data = df.isnull().sum()
if missing_data.sum() > 0:
    print(missing_data[missing_data > 0])
else:
    print("没有缺失值")

# 准备分析数据 - 只选择数值型列
print(f"\n准备进行相关性分析...")
numeric_columns = df.select_dtypes(include=[np.number]).columns.tolist()
print(f"数值型列: {numeric_columns}")

# 确保目标列在数值型列中
if target_column not in numeric_columns:
    print(f"错误: 目标列'{target_column}'不是数值型")
    exit()

# 移除目标列, 得到解释变量
explanatory_variables = [col for col in numeric_columns if col != target_column]
print(f"\n解释变量 ({len(explanatory_variables)}个):")
for i, var in enumerate(explanatory_variables, 1):
    print(f"{i:2d}. {var}")

# 获取目标变量数据
target_data = df[target_column].dropna()
print(f"\n目标变量'{target_column}'统计:")
print(f"有效数据点: {len(target_data)}")
print(f"均值: {target_data.mean():.4f}")
print(f"标准差: {target_data.std():.4f}")
print(f"最小值: {target_data.min():.4f}")
print(f"最大值: {target_data.max():.4f}")

# 计算斯皮尔曼相关系数

```

```

print(f"\n正在计算斯皮尔曼相关系数...")
correlation_results = []

for var in explanatory_variables:
    # 获取两个变量的有效数据
    var_data = df[var]
    target_var_data = df[target_column]

    # 移除任一变量为空的行
    valid_mask = var_data.notna() & target_var_data.notna()
    var_clean = var_data[valid_mask]
    target_clean = target_var_data[valid_mask]

    if len(var_clean) > 1: # 至少需要2个数据点
        try:
            correlation, p_value = spearmanr(var_clean, target_clean)
            correlation_results.append({
                '变量': var,
                '斯皮尔曼相关系数': correlation,
                'P值': p_value,
                '有效样本数': len(var_clean),
                '显著性': '显著' if p_value < 0.05 else '不显著'
            })
        except Exception as e:
            print(f"计算 {var} 的相关系数时出错: {e}")
            correlation_results.append({
                '变量': var,
                '斯皮尔曼相关系数': np.nan,
                'P值': np.nan,
                '有效样本数': len(var_clean),
                '显著性': '计算错误'
            })
    else:
        correlation_results.append({
            '变量': var,
            '斯皮尔曼相关系数': np.nan,
            'P值': np.nan,
            '有效样本数': len(var_clean),
            '显著性': '数据不足'
        })

# 创建结果DataFrame
results_df = pd.DataFrame(correlation_results)

# 按相关系数绝对值排序
results_df['相关系数绝对值'] = abs(results_df['斯皮尔曼相关系数'])
results_df = results_df.sort_values('相关系数绝对值', ascending=False)

# 显示结果
print(f"\n斯皮尔曼相关系数分析结果:")
print("="*80)
display_df = results_df[['变量', '斯皮尔曼相关系数', 'P值', '有效样本数', '显著性']].copy()
for i, row in display_df.iterrows():
    print(f"{row['变量']:<25} ")

```

```

        f"相关系数: {row['斯皮尔曼相关系数']:>8.4f} "
        f"P值: {row['P值']:>8.4f} "
        f"样本数: {row['有效样本数']:>4d} "
        f"显著性: {row['显著性']}")

# 统计显著性结果
significant_count = (results_df['P值'] < 0.05).sum()
total_count = len(results_df)
print(f"\n统计摘要:")
print(f"总变量数: {total_count}")
print(f"显著相关的变量数: {significant_count}")
print(f"显著相关比例: {significant_count/total_count*100:.1f}%")

# 找出相关性最强的变量
max_corr_idx = results_df['相关系数绝对值'].idxmax()
strongest_var = results_df.loc[max_corr_idx]
print(f"相关性最强的变量: {strongest_var['变量']} (r = {strongest_var['斯皮尔曼相关系数']:.4f})")

# 保存结果到Excel
print(f"\n正在保存结果到Excel...")
try:
    # 确保输出目录存在
    output_dir = os.path.dirname(output_excel)
    if not os.path.exists(output_dir):
        os.makedirs(output_dir)

    # 保存主要结果
    save_df = results_df[['变量', '斯皮尔曼相关系数', 'P值', '有效样本数', '显著性']].copy()
    save_df.to_excel(output_excel, index=False, sheet_name='斯皮尔曼相关系数')
    print(f"结果已保存到: {output_excel}")
except Exception as e:
    print(f"保存Excel文件时发生错误: {e}")

# 创建可视化图像
print(f"\n正在创建可视化图像...")

# 设置图像大小和布局
fig = plt.figure(figsize=(16, 12))

# 创建2x2的子图布局
gs = fig.add_gridspec(2, 2, height_ratios=[1, 1], width_ratios=[1, 1])

# 子图1: 相关系数条形图
ax1 = fig.add_subplot(gs[0, :])
valid_results = results_df.dropna(subset=['斯皮尔曼相关系数'])
colors = ['red' if r < 0 else 'blue' for r in valid_results['斯皮尔曼相关系数']]
bars = ax1.barh(range(len(valid_results)), valid_results['斯皮尔曼相关系数'], color=colors,
                alpha=0.7)
ax1.set_yticks(range(len(valid_results)))
ax1.set_yticklabels(valid_results['变量'], fontsize=10)
ax1.set_xlabel('斯皮尔曼相关系数', fontsize=12)
ax1.set_title(f'各变量与{target_column}的斯皮尔曼相关系数', fontsize=14, fontweight='bold')
ax1.axvline(x=0, color='black', linestyle='-', alpha=0.5)
ax1.grid(True, alpha=0.3)

```

```

# 在条形图上添加数值标签
for i, (bar, corr) in enumerate(zip(bars, valid_results['斯皮尔曼相关系数'])):
    if corr >= 0:
        ax1.text(corr + 0.01, bar.get_y() + bar.get_height()/2, f'{corr:.3f}',
                  va='center', ha='left', fontsize=9)
    else:
        ax1.text(corr - 0.01, bar.get_y() + bar.get_height()/2, f'{corr:.3f}',
                  va='center', ha='right', fontsize=9)

# 子图2: P值分布
ax2 = fig.add_subplot(gs[1, 0])
valid_p_values = results_df.dropna(subset=['P值'])
significant_vars = valid_p_values[valid_p_values['P值'] < 0.05]['变量']
non_significant_vars = valid_p_values[valid_p_values['P值'] >= 0.05]['变量']

# 创建P值的条形图
p_colors = ['green' if p < 0.05 else 'orange' for p in valid_p_values['P值']]
bars2 = ax2.barh(range(len(valid_p_values)), valid_p_values['P值'], color=p_colors, alpha=0.7)
ax2.set_yticks(range(len(valid_p_values)))
ax2.set_yticklabels(valid_p_values['变量'], fontsize=9)
ax2.set_xlabel('P值', fontsize=12)
ax2.set_title('各变量相关性的显著性检验', fontsize=12, fontweight='bold')
ax2.axvline(x=0.05, color='red', linestyle='--', alpha=0.8, label='显著性阈值(0.05)')
ax2.legend()
ax2.grid(True, alpha=0.3)

# 子图3: 相关系数绝对值排序
ax3 = fig.add_subplot(gs[1, 1])
top_10 = valid_results.head(10) # 显示前10个相关性最强的变量
bars3 = ax3.bar(range(len(top_10)), top_10['相关系数绝对值'],
                 color=['darkred' if r < 0 else 'darkblue' for r in top_10['斯皮尔曼相关系数']],
                 alpha=0.7)
ax3.set_xticks(range(len(top_10)))
ax3.set_xticklabels(top_10['变量'], rotation=45, ha='right', fontsize=9)
ax3.set_ylabel('相关系数绝对值', fontsize=12)
ax3.set_title('相关性最强的前10个变量', fontsize=12, fontweight='bold')
ax3.grid(True, alpha=0.3)

# 在柱状图上添加数值
for i, (bar, abs_corr) in enumerate(zip(bars3, top_10['相关系数绝对值'])):
    ax3.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.005,
              f'{abs_corr:.3f}', ha='center', va='bottom', fontsize=8)

# 调整布局
plt.tight_layout()

# 保存图像
try:
    plt.savefig(output_image, dpi=300, bbox_inches='tight', facecolor='white')
    print(f"可视化图像已保存到: {output_image}")
except Exception as e:
    print(f"保存图像时发生错误: {e}")

```



```

# 显示图像
plt.show()

# 生成分析报告
print(f"\n分析报告:")
print("="*60)
print(f"目标变量: {target_column}")
print(f"分析的解釋变量数量: {len(explanatory_variables)}")
print(f"有效的相关性分析数量: {len(valid_results)}")
print(f"\n显著相关的变量 (P < 0.05):")
significant_vars = results_df[results_df['P值'] < 0.05].sort_values('相关系数绝对值',
    ascending=False)
if len(significant_vars) > 0:
    for _, row in significant_vars.iterrows():
        correlation_strength = ""
        abs_corr = abs(row['斯皮尔曼相关系数'])
        if abs_corr >= 0.7:
            correlation_strength = "强相关"
        elif abs_corr >= 0.5:
            correlation_strength = "中等相关"
        elif abs_corr >= 0.3:
            correlation_strength = "弱相关"
        else:
            correlation_strength = "极弱相关"

        print(f" {row['变量']:<25} r = {row['斯皮尔曼相关系数']:>7.4f} ({correlation_strength})")
else:
    print(" 没有发现显著相关的变量")

print(f"\n处理完成!")
print(f"Excel结果文件: {output_excel}")
print(f"可视化图像文件: {output_image}")

```

附录 F 问题四源程序

```

#随机森林、逻辑回归、支持向量机、决策树和K紧邻模型比较
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, cross_val_score, learning_curve
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
    f1_score, roc_auc_score, roc_curve, confusion_matrix,
    classification_report)
import warnings
import os

```

```

warnings.filterwarnings('ignore')
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

# 文件路径
data_path = r"C:\Users\DELL\Desktop\25C\4.3 二进制.xlsx"
output_path = r"C:\Users\DELL\Desktop\25C"

# 数据加载
try:
    df = pd.read_excel(data_path)
except FileNotFoundError:
    # 创建模拟数据
    np.random.seed(42)
    n_samples = 1000

    df = pd.DataFrame({
        '检测孕周': np.random.normal(20, 3, n_samples),
        '孕妇BMI': np.random.normal(23, 3, n_samples),
        '原始读段数': np.random.normal(50000000, 10000000, n_samples),
        '在参考基因组上比对的比例': np.random.normal(0.95, 0.02, n_samples),
        '重复读段的比例': np.random.normal(0.15, 0.03, n_samples),
        '唯一比对的读段数': np.random.normal(40000000, 8000000, n_samples),
        'GC含量': np.random.normal(0.42, 0.02, n_samples),
        '13号染色体的Z值': np.random.normal(0, 1, n_samples),
        '18号染色体的Z值': np.random.normal(0, 1, n_samples),
        '21号染色体的Z值': np.random.normal(0, 1, n_samples),
        'X染色体的Z值': np.random.normal(0, 1, n_samples),
        'Y染色体的Z值': np.random.normal(0, 1, n_samples),
        'Y染色体浓度': np.random.normal(0.1, 0.05, n_samples),
        'X染色体浓度': np.random.normal(0.5, 0.1, n_samples),
        '13号染色体的GC含量': np.random.normal(0.38, 0.02, n_samples),
        '18号染色体的GC含量': np.random.normal(0.39, 0.02, n_samples),
        '21号染色体的GC含量': np.random.normal(0.41, 0.02, n_samples),
        '被过滤掉读段数的比例': np.random.normal(0.05, 0.01, n_samples),
    })

    df['染色体的非整倍体'] = (
        (df['21号染色体的Z值'] > 3) |
        (df['18号染色体的Z值'] > 3) |
        (df['13号染色体的Z值'] > 3) |
        np.random.choice([0, 1], n_samples, p=[0.95, 0.05])
    ).astype(int)

# 数据预处理
feature_columns = [col for col in df.columns if col != '染色体的非整倍体']
X = df[feature_columns].fillna(df[feature_columns].mean())
y = df['染色体的非整倍体']

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X_scaled = pd.DataFrame(X_scaled, columns=feature_columns)

```

```

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42, stratify=y
)

# 定义模型
models = {
    '随机森林': RandomForestClassifier(n_estimators=100, random_state=42),
    '逻辑回归': LogisticRegression(random_state=42, max_iter=1000),
    '支持向量机': SVC(probability=True, random_state=42),
    '决策树': DecisionTreeClassifier(random_state=42),
    'K近邻': KNeighborsClassifier(n_neighbors=5)
}

# 训练和评估模型
results = {}
predictions = {}
probabilities = {}

for name, model in models.items():
    model.fit(X_train, y_train)

    y_pred = model.predict(X_test)
    y_pred_proba = model.predict_proba(X_test)[:, 1]

    accuracy = accuracy_score(y_test, y_pred)
    precision = precision_score(y_test, y_pred)
    recall = recall_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)
    auc = roc_auc_score(y_test, y_pred_proba)

    cv_scores = cross_val_score(model, X_train, y_train, cv=5, scoring='accuracy')

    results[name] = {
        'accuracy': accuracy,
        'precision': precision,
        'recall': recall,
        'f1_score': f1,
        'auc': auc,
        'cv_mean': cv_scores.mean(),
        'cv_std': cv_scores.std()
    }

    predictions[name] = y_pred
    probabilities[name] = y_pred_proba

# 保存结果
results_df = pd.DataFrame(results).T.round(4)

os.makedirs(output_path, exist_ok=True)
results_path = os.path.join(output_path, "4.4结果.xlsx")

with pd.ExcelWriter(results_path) as writer:
    results_df.to_excel(writer, sheet_name='模型性能比较')

```

```

for name in models.keys():
    report = classification_report(y_test, predictions[name], output_dict=True)
    report_df = pd.DataFrame(report).T
    report_df.to_excel(writer, sheet_name=f'{name}_详细报告')

# 可视化
viz_path = os.path.join(output_path, "可视化图像")
os.makedirs(viz_path, exist_ok=True)

# ROC曲线比较
plt.figure(figsize=(12, 8))
colors = ['blue', 'red', 'green', 'orange', 'purple']

for i, (name, model) in enumerate(models.items()):
    fpr, tpr, _ = roc_curve(y_test, probabilities[name])
    auc = results[name]['auc']
    plt.plot(fpr, tpr, color=colors[i], linewidth=2,
             label=f'{name} (AUC = {auc:.3f})')

plt.plot([0, 1], [0, 1], 'k--', linewidth=1, label='随机分类器')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('假阳性率 (FPR)')
plt.ylabel('真阳性率 (TPR)')
plt.title('ROC曲线比较')
plt.legend(loc="lower right")
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig(os.path.join(viz_path, 'ROC曲线比较.png'), dpi=300, bbox_inches='tight')
plt.show()

# 模型性能指标比较
fig, axes = plt.subplots(2, 2, figsize=(15, 12))
metrics = ['accuracy', 'precision', 'recall', 'f1_score']
metric_names = ['准确率', '精确率', '召回率', 'F1分数']

for idx, (metric, metric_name) in enumerate(zip(metrics, metric_names)):
    ax = axes[idx//2, idx%2]
    values = [results[name][metric] for name in models.keys()]
    bars = ax.bar(models.keys(), values, color=colors)
    ax.set_title(f'{metric_name}比较')
    ax.set_ylabel(metric_name)
    ax.set_ylim(0, 1)

    for bar, value in zip(bars, values):
        ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.01,
                f'{value:.3f}', ha='center', va='bottom')

    plt.setp(ax.get_xticklabels(), rotation=45, ha='right')

plt.tight_layout()
plt.savefig(os.path.join(viz_path, '模型性能指标比较.png'), dpi=300, bbox_inches='tight')
plt.show()

```

```

# 特征重要性分析
rf_model = models['随机森林']
feature_importance = pd.DataFrame({
    'feature': feature_columns,
    'importance': rf_model.feature_importances_
}).sort_values('importance', ascending=True)

plt.figure(figsize=(10, 8))
plt.barh(range(len(feature_importance)), feature_importance['importance'])
plt.yticks(range(len(feature_importance)), feature_importance['feature'])
plt.xlabel('特征重要性')
plt.title('随机森林模型 - 特征重要性分析')
plt.tight_layout()
plt.savefig(os.path.join(viz_path, '特征重要性分析.png'), dpi=300, bbox_inches='tight')
plt.show()

# 混淆矩阵可视化
fig, axes = plt.subplots(2, 3, figsize=(18, 12))
axes = axes.ravel()

for idx, (name, model) in enumerate(models.items()):
    cm = confusion_matrix(y_test, predictions[name])
    sns.heatmap(cm, annot=True, fmt='d', ax=axes[idx], cmap='Blues')
    axes[idx].set_title(f'{name} - 混淆矩阵')
    axes[idx].set_xlabel('预测标签')
    axes[idx].set_ylabel('真实标签')

if len(models) < 6:
    for idx in range(len(models), 6):
        fig.delaxes(axes[idx])

plt.tight_layout()
plt.savefig(os.path.join(viz_path, '混淆矩阵比较.png'), dpi=300, bbox_inches='tight')
plt.show()

# 数据分布可视化
plt.figure(figsize=(15, 10))

plt.subplot(2, 3, 1)
y.value_counts().plot(kind='bar')
plt.title('染色体非整倍体分布')
plt.ylabel('样本数量')

top_features = feature_importance.tail(5)['feature'].values
for idx, feature in enumerate(top_features):
    plt.subplot(2, 3, idx+2)
    plt.hist(df[df['染色体的非整倍体']==0][feature], alpha=0.5, label='正常', bins=30)
    plt.hist(df[df['染色体的非整倍体']==1][feature], alpha=0.5, label='异常', bins=30)
    plt.title(f'{feature}分布')
    plt.legend()

plt.tight_layout()
plt.savefig(os.path.join(viz_path, '数据分布分析.png'), dpi=300, bbox_inches='tight')
plt.show()

```

```

# 交叉验证结果可视化
plt.figure(figsize=(12, 6))
cv_means = [results[name]['cv_mean'] for name in models.keys()]
cv_stds = [results[name]['cv_std'] for name in models.keys()]

plt.errorbar(range(len(models)), cv_means, yerr=cv_stds,
             fmt='o-', capsize=5, capthick=2, linewidth=2)
plt.xticks(range(len(models)), models.keys(), rotation=45)
plt.ylabel('交叉验证准确率')
plt.title('5折交叉验证结果比较')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig(os.path.join(viz_path, '交叉验证结果.png'), dpi=300, bbox_inches='tight')
plt.show()

# 学习曲线分析
def plot_learning_curve(estimator, title, X, y, cv=None, train_sizes=np.linspace(.1, 1.0, 5)):
    plt.figure(figsize=(10, 6))
    plt.title(title)
    plt.xlabel("训练样本数")
    plt.ylabel("准确率")

    train_sizes, train_scores, test_scores = learning_curve(
        estimator, X, y, cv=cv, train_sizes=train_sizes)

    train_scores_mean = np.mean(train_scores, axis=1)
    train_scores_std = np.std(train_scores, axis=1)
    test_scores_mean = np.mean(test_scores, axis=1)
    test_scores_std = np.std(test_scores, axis=1)

    plt.grid()
    plt.fill_between(train_sizes, train_scores_mean - train_scores_std,
                    train_scores_mean + train_scores_std, alpha=0.1, color="r")
    plt.fill_between(train_sizes, test_scores_mean - test_scores_std,
                    test_scores_mean + test_scores_std, alpha=0.1, color="g")
    plt.plot(train_sizes, train_scores_mean, 'o-', color="r", label="训练集得分")
    plt.plot(train_sizes, test_scores_mean, 'o-', color="g", label="交叉验证得分")

    plt.legend(loc="best")
    return plt

# 最佳模型学习曲线
best_model_name = max(results.keys(), key=lambda x: results[x]['auc'])
best_model = models[best_model_name]

plot_learning_curve(best_model, f'{best_model_name} - 学习曲线', X_scaled, y, cv=5)
plt.savefig(os.path.join(viz_path, f'{best_model_name}_学习曲线.png'), dpi=300,
            bbox_inches='tight')
plt.show()

# 简要结果输出
print(f"数据形状: {df.shape}")
print(f"最佳模型: {best_model_name}")

```

```
print(f"最佳AUC: {max(results[name]['auc'] for name in results.keys()):.4f}")

print(f"\n模型性能:")
for name in models.keys():
    print(f"{name}: AUC={results[name]['auc']:.4f}, F1={results[name]['f1_score']:.4f}")

print(f"\n结果保存至: {results_path}")
print(f"图表保存至: {viz_path}")
```